QUANT RESEARCH · V1.0 · 2026

```
> scanner.init()
> fetch_events(NBA)
> detect_arb() ...
  > edge = 87bps
> execute(YES, NO)
  > pnl += $14.32
  > _loop_
```

Sports Arbitrage on Polymarket A Quant Dev Masterclass

Single-market YES+NO edges, combinatorial covering portfolios,
and production scanner architecture for NBA-scale volume.

A complete operator's manual for the sports prediction-market edge: the mathematics of binary and NegRisk arbitrage, the early-2026 fee regime, real-time Gamma and CLOB data pipelines, an LP-based covering portfolio solver, Kelly sizing under settlement risk, and a fully async execution engine with partial-fill hedging. Twelve runnable Python modules ship alongside this guide.

~100	$0.60	0.75%
OPPS / MARKET / DAY	MEDIAN PROFIT / $1	TAKER FEE (MAKER = 0)

PREPARED FOR
Quant developers & arb operators
NBA-focused edition

VERSION
v1.0 · 2026
12 Python modules included

POLYGON · USDC · GAMMA-API · CLOB-WS

LIVE

Table of Contents

CHAPTER 0 · FRAMING

Why Sports, Not Politics

Every new arbitrage developer who lands on Polymarket points their scanner at the same target first: politics. Elections, confirmation hearings, geopolitical markets. The instinct is rational — those are the markets with eight-figure volume headlines, the markets whose screenshots go viral on Crypto Twitter, the markets the platform itself promotes on its homepage. This chapter argues that the instinct is wrong, and that the wallets quietly compounding month after month are pointed somewhere else entirely. We will back the argument with data from a year-long academic measurement of Polymarket arbitrage, explain why the structural composition of sports flow keeps producing repeatable edges, and end with the operational implication: your scanner architecture should be optimized for the nightly cadence of professional sports, not the once-a-year cadence of an election cycle.

0.1 What the data actually shows

The clearest measurement of Polymarket arbitrage activity to date is the paper *Arbitrage Opportunities on Polymarket* (arXiv:2508.03474), which analyzes roughly a full year of platform-wide price data and tracks which opportunities real wallets exploited, not just which ones existed in theory. Three findings from that paper reframe the entire strategy space.

Finding 1 — Politics has the fattest single payouts. The largest individual arbitrage conditions the paper measured were predominantly in politics, mostly inside multi-outcome NegRisk markets tied to the 2024 US election cycle. Billions of dollars of volume, thousands of conditions, prices moving every hour. When new developers read about Polymarket arbitrage, this is the picture they form in their head.

Finding 2 — Sports has the most opportunities by count. Inside multi-outcome markets the paper measured around one hundred arbitrage opportunities per market on average, and the biggest outliers sat in sports. The same nightly cadence that produces thin per-opportunity edges also produces enormous opportunity counts. Politics gives you a few fat conditions per cycle; sports gives you a hundred thin conditions per game, every night, across a dozen leagues.

Finding 3 — Real money was taken mostly from sports single-condition arbitrage. This is the line most readers skip. The paper tracked which opportunities real wallets actually exploited, as opposed to which ones existed on paper. For single-condition arbitrage (YES and NO of one outcome drifting below or above a dollar), sports markets dominated the exploited profit so heavily that they surpassed the entire 2024 election cycle in that category. Meanwhile only about one percent of the theoretically identified election opportunities were ever exploited at all. The politics opportunities are huge on paper and mostly left on the table; the sports opportunities are smaller individually, taken constantly, and add up to more.

> *** KEY INSIGHT**
> Politics gives you the screenshot. Sports gives you the paycheck. The headline numbers will always point at elections because that is where the volume photo-ops are. The wallets quietly printing month after month are pointed at the games.

0.2 Politics arbitrage is a seasonal market

The fat political numbers in the paper came from one window: the election. Elections do not happen every week. You get a presidential cycle, a few primaries, the occasional geopolitical market, then long stretches of nothing. A scanner pointed at politics waits, and waits, and waits. Capital sits cold while you wait for the next headline to create enough volatility to throw prices off a dollar. Idle capital earns nothing.

This is the trap. You optimized your entire bot for the loudest market on the platform, and that market only pays a few weeks a year. Event-driven means seasonal; seasonal means idle. The operational cost of running a trading system (VPS, monitoring, developer attention, capital lockup) does not disappear during the idle months. A politics-only scanner carries 100% of its infrastructure cost against a fraction of the year of revenue.

0.3 Sports is the nightly edge

Now look at the other side. NBA tonight. NFL on Sunday. Soccer every day across a dozen leagues. Tennis, MMA, the whole calendar. Every single night a fresh batch of markets opens, fills with retail flow, and resolves within hours. Politics gives you one giant volatile window a year. Sports gives you a new volatile window every night. That is the difference between a strategy that prints in bursts and one that prints on a schedule.

Small edge, many shots, every night — that is the compounding game, and sports is built for it. The mathematical edge per opportunity is smaller in sports than in politics, but the law of large numbers applies cleanly because the opportunity count is so high. Variance of monthly PnL drops fast when you take a hundred small edges instead of two large ones.

0.4 Two different arb types live in two different places

This is the part most developers skip past, and it is where running one bot pointed at one market type leaves half the edge on the floor. There are two structurally different arbitrage types on Polymarket, and they concentrate in different market sectors.

Single-condition arbitrage — YES and NO of one outcome summing below or above a dollar. This is the high-frequency, every-night game. Per the paper, this profit concentrates in sports. Point a single-condition YES+NO scanner at the daily sports flow for your bread and butter.

Within-market rebalancing arbitrage — the sum of all YES tokens across a multi-outcome NegRisk market drifting off a dollar. The paper found this profit concentrated heavily in politics. Sports was almost absent from it. So when a real event window opens — an election, a primary, a confirmation fight — turn on a NegRisk rebalancing scanner pointed at politics. Different market, different arb, different target.

> *** KEY INSIGHT**
> One bot pointed at one market type leaves half the edge on the floor. Match the scanner to the market type: single-condition YES+NO on the daily sports flow, NegRisk rebalancing on politics only when an event window opens.

0.5 Why sports stays inefficient

The deeper reason sports keeps paying is who trades it. Politics arbitrage gets picked over by smart money: quants, funds, and bot squads all camp the election. Sports gets retail. People bet their team with their heart. They pile into a YES on emotion the second their guy scores. Then an injury hits, a ref call lands, momentum flips, and the price lags the new reality for a few seconds while the order book catches up. That lag is your gap.

The paper even found sports markets are often overvalued, with long opportunities dominating — a sign the crowd keeps overpaying one side. And the mispricing is not small. Median profit per dollar across these single-condition opportunities sat around sixty cents in the pre-fee measurement window. That is not a three-cent skim; that is a market that is openly wrong, over and over, in a place nobody serious is watching.

One caveat the paper itself flags: that sixty-cent figure was measured before Polymarket introduced fees. The mispricing is still there. What changed is what you net after you trade it. We cover the fee math in Chapter 1 and the maker-versus-taker workaround in Chapter 4.

0.6 The 2026 fee change and the maker pivot

Sports received a taker fee in early 2026, and most categories followed. The opportunities are still where the data says they are (sports, every night), but you cannot simply take them anymore. Take the gap as a taker and the fee bites hardest near 50/50, exactly where these sports gaps sit. A 0.75% taker fee on both legs of a YES+NO arb at 50/50 prices eats 1.5 cents per dollar of payout, which is more than many sports gaps gross.

The fix is to capture them as a maker. Rest a limit order, get filled, pay zero. The edge the paper identified is still real; you just have to collect it the right way now. This is also why the default min-edge thresholds in the code accompanying this guide are split: 75 bps for taker execution (must clear the fee exactly) and 30 bps for maker execution (only need a slippage buffer, since fee is zero).

0.7 How to actually point your scanner

Concrete setup, no fluff:

- **Filter the CLOB for sports markets resolving today.** You want fresh, high-turnover books, not stale weeklies. The code defaults to a 12-hour resolution window.
- **Watch the YES+NO sum on single conditions.** Flag the moment it drifts past your threshold. With new taker fees peaking near 50/50, set the threshold higher in coin-flip markets than you would have a year ago.
- **Be the maker, not the taker.** Rest your limit orders into the gap instead of crossing the spread. As a maker you pay zero fee and the threshold math goes back in your favor.
- **Bias toward the overvalued side.** Sports runs rich, so the short / sell-above-a-dollar leg shows up more than people expect.
- **Size for liquidity.** Sports books are thinner than the election was, so your fills are smaller. Do not model election-size depth onto a Tuesday NBA game.
- **Run it every night.** The whole point of sports is frequency. A scanner that only wakes up for elections is a politics bot wearing a sports jersey.

0.8 How to read this guide

The remainder of this guide is structured for a working quant developer who wants both the math and the code. **Chapter 1** is the formal mathematics: implied probabilities, overround, the four arbitrage conditions (single-market, multi-outcome rebalance, combinatorial covering, cross-platform), fee impact, and the risk-free versus near-risk-free distinction. **Chapter 2** covers the Polymarket API stack (Gamma, CLOB REST, CLOB WebSocket) and how to discover and group related markets on one event. **Chapter 3** is the full scanner implementation with line-by-line code listings for twelve production modules. **Chapter 4** covers operational hardening: low-latency setup, slippage modeling, in-play windows, monitoring, scaling, and compliance. **Chapter 5** closes with edge maintenance, failure modes, and hybrid strategies. Two appendices follow: a code module index and a condensed mathematical reference.

All Python code shown in this PDF is delivered as runnable modules in the companion package. The PDF is the theory and walkthrough; the modules are the implementation. Read them in parallel.

CHAPTER 1 · MATHEMATICS

Core Concepts & Mathematics

This chapter develops the formal mathematics of prediction-market arbitrage from first principles. We start with the microstructure of binary outcome tokens on Polymarket, derive the implied probability and overround formulas, then build up four distinct arbitrage conditions in order of increasing complexity: single-market YES+NO arbitrage, multi-outcome NegRisk rebalancing, combinatorial cross-market covering portfolios, and cross-platform arbitrage against traditional sportsbooks. We close with a rigorous treatment of the early-2026 fee regime and a taxonomy of which strategies are truly risk-free versus merely edge-adjusted.

1.1 Prediction market microstructure

Polymarket is a centralized limit order book (CLOB) for binary outcome tokens settled on the Polygon blockchain. Each market poses a yes/no question — for example, *Will the Lakers defeat the Celtics on January 15?* — and mints two ERC-1155 tokens: a YES token and a NO token. The collateral is USDC on Polygon. At resolution, exactly one token redeems for $1 of USDC and the other redeems for $0. The settlement logic uses the UMA optimistic oracle for dispute resolution.

The two-outcome redemption rule has a critical mathematical consequence: one YES token plus one NO token is always worth exactly $1, regardless of outcome. This is the foundation of every arbitrage strategy on the platform. If you can acquire one share of each side for a total cost strictly less than $1 (after fees), you have locked in a guaranteed profit. The art is in finding situations where the market lets you do this, and the rest of this chapter formalizes when and how.

NegRisk multi-outcome markets extend this structure to events with more than two mutually exclusive outcomes — for example, *Who wins the 2025 NBA MVP?* with twenty candidates. Rather than spawning twenty independent binary markets (which would each need their own order book), Polymarket uses the NegRisk framework: a single ERC-1155 contract holds one YES token per outcome, with a shared collateral pool. Holding one share of every YES token still guarantees a $1 payout, because exactly one outcome will resolve YES. This shared-collateral property is what makes NegRisk rebalancing arbitrage possible: the sum of all YES prices should theoretically equal $1, and when it drifts, you can rebalance.

1.2 Implied probability and overround

The price of a YES token on Polymarket is a probability expressed in dollars. A YES price of $0.65 implies the market assigns a 65% probability to the YES outcome. Formally, given a YES ask price P_yes (the cost to buy one YES share) and a NO ask price P_no (the cost to buy one NO share):

$$p\_implied(YES) = P\_yes \,/\, \$1 = P\_yes$$

$$p\_implied(NO) = P\_no \,/\, \$1 = P\_no$$

In a perfectly efficient market with no fees, the law of one price demands that P_yes + P_no = $1, because holding one of each guarantees a $1 payout. The **overround** (also called the vig or hold) measures how far the sum deviates from $1:

$$overround = (P\_yes + P\_no) - \$1$$

An overround greater than zero means the market is overpriced — the sum of asks exceeds $1, so buying both sides loses money. An overround less than zero means an arbitrage exists — the sum of asks is below $1, so buying both sides locks in profit. Most of the time in a healthy two-sided market, the overround is slightly positive because of the bid-ask spread: P_yes_ask + P_no_ask = P_yes_ask + (1 − P_yes_bid) = 1 + spread_yes > 1.

Normalized implied probabilities remove the overround to recover the market's true probability estimate. Given overround r = P_yes + P_no − 1:

$$p\_norm(YES) = P\_yes / (P\_yes + P\_no)$$

$$p\_norm(NO) = P\_no / (P\_yes + P\_no)$$

These normalized probabilities are what you compare against external sources (sportsbook odds, your own model) when looking for cross-platform or model-driven edges. The unnormalized prices are what you trade.

Worked example: Lakers vs Celtics moneyline

Consider an NBA moneyline market: *Will the Lakers defeat the Celtics?* The order book shows YES best ask at $0.48 and NO best ask at $0.49 (which corresponds to a YES best bid of $0.51).

Implied probabilities: p(YES) = 0.48, p(NO) = 0.49. Sum = 0.97, overround = −$0.03. Normalized probabilities: p_norm(YES) = 0.48 / 0.97 = 0.4948, p_norm(NO) = 0.49 / 0.97 = 0.5052. The market is essentially a coin flip with a slight lean toward the Celtics, but the unnormalized prices sum to less than $1, which means an arbitrage exists. We will compute the exact profit after fees in the next section.

1.3 Single-market arbitrage

Single-market arbitrage is the simplest and most repeatable strategy: buy one YES and one NO of the same binary market, and if the combined cost is less than $1 after fees, you have locked in a riskless profit. Per the article framing this guide, this is the bread-and-butter strategy for sports — the high-frequency, every-night edge that real wallets exploit more than any other arbitrage type on Polymarket.

Arbitrage condition

Let P_yes_ask be the best ask price for YES, P_yes_bid the best bid for YES. The cost to buy one YES share is P_yes_ask. The cost to buy one NO share is the NO ask, which equals 1 − P_yes_bid (because buying NO at the NO ask is equivalent to selling YES at the YES bid, modulo minting mechanics). Define the gross cost:

$$C\_gross = P\_yes\_ask + (1 - P\_yes\_bid) = 1 + spread\_yes$$

Without fees, this gross cost always equals 1 + spread_yes, which is greater than $1 in any market with a positive bid-ask spread. The arbitrage only appears when the spread inverts — that is, when P_yes_bid > P_yes_ask, which cannot happen in a single order book, OR when the YES and NO prices come from different liquidity sources that have not yet synchronized. The latter is what creates sports single-market arbitrage in practice: a taker crosses the spread on one side, a market maker pulls quotes on the other, and for a few seconds the best bid for YES exceeds the best ask for NO (equivalently, the sum P_yes_ask + P_no_ask drops below $1).

Formally, the arbitrage condition (before fees) is:

$$P\_yes\_ask + P\_no\_ask < \$1$$

Including the taker fee f (currently 0.0075 = 75 bps per leg), the net cost per share pair is:

$$C\_net = (P\_yes\_ask + P\_no\_ask) \cdot (1 + f)$$

And the arbitrage exists when C_net < $1, equivalently when (P_yes_ask + P_no_ask) < 1 / (1 + f). At f = 0.0075, the threshold sum is approximately $0.9926 — meaning the raw YES+NO sum must drop below roughly 99.26 cents to clear fees as a taker. The guaranteed profit per share pair is:

$$\pi = \$1 - C\_net = \$1 - (P\_yes\_ask + P\_no\_ask) \cdot (1 + f)$$

Position sizing for equal payout

If you buy N_yes shares of YES at P_yes_ask and N_no shares of NO at P_no_ask, the payout is max(N_yes, N_no) · $1 (whichever side wins). To guarantee equal payout regardless of outcome, you need N_yes = N_no = N. Total cost is then N · (P_yes_ask + P_no_ask) · (1 + f), guaranteed payout is N · $1, and profit scales linearly with N. The maximum N is constrained by the smaller of the two sides' available depth at the ask.

Maker execution: zero-fee arbitrage

The early-2026 fee change introduced a 0.75% taker fee but kept the maker fee at zero. As a maker, you rest limit orders on the book rather than crossing the spread. If both your YES bid and your NO bid (equivalently, your YES ask at 1 − P_no_bid) fill, the total cost is:

$$C\_maker = P\_yes\_bid\_rested + P\_no\_bid\_rested$$

with zero fee. If you can rest both sides such that P_yes_bid_rested + P_no_bid_rested < $1, every pair that fills locks in pure profit. The trade-off is execution risk: maker orders may not fill (especially in fast-moving sports markets), and the window during which the inverted spread persists may close before both legs fill. The default scanner configuration prefers maker execution with an 8-second timeout, falling back to taker if the maker order has not filled by then.

Worked example with fees

Returning to the Lakers/Celtics moneyline: YES ask = $0.48, NO ask = $0.49 (so YES bid = $0.51, NO bid = $0.52). Sum of asks = $0.97.

Taker path: C_net = $0.97 × (1 + 0.0075) = $0.97 × 1.0075 = $0.9773. Profit per share pair = $1 − $0.9773 = $0.0227. On $1,000 notional (about 1,031 share pairs), gross profit ≈ $23.40. Net of fees that is the realized profit, since fees are already in C_net. Edge in basis points: $0.0227 / $0.97 = 234 bps gross, 161 bps net of fees.

Maker path: Rest a YES bid at $0.47 and a NO bid at $0.48 (equivalently, a YES ask at $0.52). If both fill, C_maker = $0.47 + $0.48 = $0.95. Profit per share pair = $1 − $0.95 = $0.05. Edge = 526 bps. The maker path captures roughly 2.2× the taker edge on the same opportunity, but you must wait for both legs to fill and you carry the risk that only one fills before the spread closes.

1.4 Multi-outcome NegRisk rebalancing arbitrage

NegRisk markets have N ≥ 3 mutually exclusive outcomes. Each outcome has its own YES token, and the prices should theoretically sum to $1 because holding one share of every YES guarantees $1 payout (exactly one outcome wins). When the sum drifts off $1, rebalancing arbitrage becomes possible.

Buy-all-YES arb condition

If Σ_i P_yes_ask(i) < $1, you can buy one share of every YES and lock in a guaranteed $1 payout. Including fees:

$$C\_net\_buy\_all = (\Sigma\_i\ P\_yes\_ask(i)) \cdot (1 + f)$$

$$\pi\_buy\_all = \$1 - C\_net\_buy\_all$$

The arbitrage exists when Σ_i P_yes_ask(i) < 1 / (1 + f). Sizing is constrained by the minimum available ask depth across all N outcomes, because you must buy equal quantities of each to guarantee the $1 payout.

Buy-all-NO arb condition (sell-all-YES equivalent)

If Σ_i P_yes_bid(i) > $1, equivalently Σ_i (1 − P_yes_bid(i)) < N − 1, the YES side is collectively overpriced. Polymarket does not support native shorting, but you can achieve the equivalent by buying NO on every outcome. The cost of buying one NO share on each outcome is Σ_i (1 − P_yes_bid(i)) = N − Σ_i P_yes_bid(i). The payout is (N − 1) · $1, because all NOs except the winner's NO pay out. Profit:

$$\pi\_sell\_all = (N - 1) - (N - \Sigma\ P\_yes\_bid) \cdot (1 + f) = \Sigma\ P\_yes\_bid - 1 - fee\_term$$

The arbitrage exists when Σ_i P_yes_bid(i) > 1 + fee_buffer. Per the article, this NegRisk rebalancing profit concentrates heavily in politics and is almost absent from sports — so this detector is included in the code for completeness but is not the primary sports edge.

1.5 Combinatorial cross-market arbitrage

Combinatorial arbitrage is the deepest edge in the strategy space and the one most readers underuse. The idea: a single sports game spawns multiple logically dependent markets — moneyline, spread, totals, player props, quarter lines. These markets share an underlying event, so their outcomes are not independent. By constructing a portfolio of positions across these markets, you can sometimes find a basket that pays at least $1 in every possible atomic outcome, for a total cost below $1.

Atomic outcome enumeration

Consider an NBA game with three binary markets: moneyline (Lakers vs Celtics), spread (Lakers −3.5), and total (O/U 220.5). Naive cross-product gives 2 × 2 × 2 = 8 atomic outcomes, but not all are physically possible. For example, the combination *Lakers win AND Lakers cover −3.5 AND under 220.5* is physically possible (Lakers win by 4+ in a low-scoring game), but *Celtics win AND Lakers cover −3.5* is impossible (if Celtics win, Lakers cannot have covered a negative spread).

Formally, an **atomic outcome** j is a fully-specified combination of results across all related markets: j ∈ {win/loss, cover/no-cover, over/under, ...}. The set of feasible atomic outcomes J is the subset of the cross-product that is physically possible given the sport's rules. Pruning infeasible outcomes is critical: if you do not prune, the LP solver will waste effort covering outcomes that can never occur and may report arbitrage that

does not exist in reality.

Payoff matrix

Define the set of candidate positions P. For each market m in the event, you can either buy YES or buy NO, giving two candidate positions per market: P = {(m_1, YES), (m_1, NO), (m_2, YES), (m_2, NO), ...}. For each position p ∈ P and each atomic outcome j ∈ J, define the payoff:

$$A[j, p] = 1 \text{ if position } p \text{ wins in outcome } j, \text{ else } 0$$

The cost of position p is its best ask price c_p (for YES) or (1 − best_bid) (for NO). The covering portfolio problem is then a linear program.

LP formulation

Variables: x_p ≥ 0 for each position p (number of shares to buy).

$$\text{minimize } \Sigma_p \, c_p \cdot x_p$$

$$\text{subject to } \Sigma_p \, A[j, p] \cdot x_p \geq 1 \text{ for each atomic outcome } j \in J$$

$$x_p \geq 0 \text{ for each position } p$$

If the optimal cost C* is less than $1 (after fee adjustment), an arbitrage exists. The guaranteed profit per unit of covering is $1 − C*. The LP finds the minimum-cost basket that pays at least $1 in every feasible atomic outcome — it is the formal definition of a riskless covering portfolio.

The dual of this LP has an elegant interpretation: dual variables y_j ≥ 0 represent a probability distribution over atomic outcomes. By strong duality, if C* < $1 then there exists no probability distribution over outcomes under which the current prices are expected-value-neutral. Equivalently, the market is collectively mispriced relative to any feasible outcome distribution.

Worked NBA example

Lakers vs Celtics. Markets: (1) Moneyline: Lakers YES @ $0.55, NO @ $0.45. (2) Spread Lakers −3.5: YES @ $0.48, NO @ $0.52. (3) Total O/U 220.5: Over @ $0.50, Under @ $0.50.

Feasible atomic outcomes (after pruning impossible combinations): (a) Lakers win by 4+ → moneyline=Lakers, spread=Lakers cover, total either; (b) Lakers win by 1-3 → moneyline=Lakers, spread=Celtics cover, total either; (c) Celtics win → moneyline=Celtics, spread=Celtics cover, total either. Total of 6 feasible atomic outcomes (3 win-margin scenarios × 2 total scenarios).

Running the LP solver (see optimizer.py) on this snapshot would return the minimum-cost basket — for example, buying moneyline-Lakers YES, spread-Celtics YES, and under-220.5 YES, with the share allocation that minimizes total cost while guaranteeing ≥$1 in each of the 6 outcomes. Whether an actual arbitrage exists depends on the live prices at scan time; the above snapshot is illustrative.

1.6 Cross-platform arbitrage

Cross-platform arbitrage compares Polymarket prices to those of traditional sportsbooks (Pinnacle, Betfair, Circa, DraftKings). The two venue types have different microstructure: Polymarket is an open CLOB with a 0.75% taker fee; sportsbooks are quote-driven with vig baked into the odds. The strategy: identify a market where Polymarket's implied probability diverges from a sportsbook's vig-free implied probability by more than the combined fee buffer.

Odds conversion

Sportsbook odds come in three formats. Decimal odds d imply a probability of 1/d. American odds +a imply probability 100/(a+100); American odds −a imply probability a/(a+100). Polymarket prices are already decimal: P_yes = decimal odds d_yes = 1/p_yes. To remove vig from a sportsbook line, normalize:

$$p\_vigfree(YES) = p\_implied(YES) \,/\, (p\_implied(YES) + p\_implied(NO))$$

For a typical NBA spread at Pinnacle with −110 on both sides, p_implied = 0.524 on each side, sum = 1.048, so the vig is 4.8% and p_vigfree = 0.5 on each side. This normalized probability is what you compare against the Polymarket price.

Edge calculation

If Polymarket YES price P_yes is below the sportsbook's vig-free probability p_vigfree by more than the fee buffer, you have an edge by buying YES on Polymarket (the cheaper venue) and either laying the bet on the sportsbook or simply taking the Polymarket side as a directional bet (unhedged, since the sportsbook side locks capital at the sportsbook).

$$edge = p\_vigfree - P\_yes$$

$$net\_edge = edge - taker\_fee - sportsbook\_vig/2$$

Settlement risk is the operational difference: Polymarket settles via UMA oracle, which can take hours to days for disputed markets, while sportsbooks typically grade within hours of game completion. Capital on the losing side is locked until both venues settle, which affects capital efficiency and should be priced into the edge threshold.

1.7 Fee structure impact analysis

Polymarket's early-2026 fee change is the single most important structural shift in the platform's history for arbitrage operators. Pre-fee, the breakeven threshold for single-market arbitrage was simply P_yes_ask + P_no_ask < $1. Post-fee, the threshold depends on execution mode.

Execution mode	Break-even sum threshold	Implied edge per leg	Total fee on cost
Taker (both legs)	$1 / (1 + 0.0075) ≈ $0.9926	74 bps per leg	0.75% × 2 = 1.5% on cost
Maker (both legs)	$1.0000 (no fee)	0 bps	0% (maker fee = 0)
Mixed maker+taker	Depends on which leg crosses	Asymmetric	Half of taker fee on average

Table 1.1 — Fee break-even thresholds by execution mode (early-2026 fee regime)

The fee bites hardest near 50/50 prices, which is exactly where most sports gaps sit. A 1-cent gap at 50/50 prices is 100 bps gross, but after 75 bps × 2 = 150 bps of taker fees, you net negative 50 bps. The same 1-cent gap at 90/10 prices is also 100 bps gross but the dollar fee is identical, so it nets negative 50 bps as well. The lesson: taker execution at 50/50 needs at least 1.5 cents of gap to break even, which is rare in liquid sports markets.

This is why maker execution became mandatory post-fee. As a maker, you pay zero fee and the threshold math resets to the pre-fee regime — any spread inversion captures profit. The trade-off is execution risk: maker orders may not fill, and the inversion window may close before both legs fill. Chapter 4 covers the operational techniques for maximizing maker fill rates.

1.8 Risk-free versus near-risk-free

Not all arbitrage is created equal. A rigorous taxonomy of risk helps you size positions correctly and avoid surprises. The four categories below span from truly riskless to merely edge-adjusted.

Category 1: Truly riskless (mathematical arbitrage)

Single-market YES+NO arb where both legs fill simultaneously, settlement is unambiguous, and no fees apply (maker execution). Example: you rest a YES bid and a NO bid that both fill within the same scan cycle on a market with a clear resolution rule (e.g., NBA moneyline). Risk: essentially zero. Position size: limited only by available depth and your bankroll.

Category 2: Near-riskless (settlement risk only)

Same as Category 1 but with non-zero settlement risk: the market could enter UMA dispute, the resolution criteria could be ambiguous (e.g., rain-delayed MLB games, NFL overtime edge cases), or the oracle could deliver an unexpected result. Position size: apply a haircut (typically 10-25% Kelly) to account for the small probability of total loss.

Category 3: Edge-adjusted (execution risk)

Combinatorial arbitrage where multiple legs must fill across multiple markets. If one leg fails to fill, you are left with unhedged directional exposure. Position size: small Kelly fraction (25%), strict execution-timeout discipline, and a rollback plan to flatten partial fills. Chapter 3 covers the execution engine that handles this.

Category 4: Directional with edge (cross-platform)

Cross-platform arbitrage where you take one side on Polymarket and the other on a sportsbook. Even if the edge is positive in expectation, you carry venue risk (one venue could fail to settle), timing risk (the two venues settle at different times), and capital-lockup risk. Position size: full Kelly with a per-position cap, and only deploy capital you can afford to have locked for the full settlement window.

> **! WARNING**
> Settlement risk is the silent killer of arbitrage strategies. Even a 1% probability of total loss per position — say, a UMA dispute that resolves against you — turns a 100 bps expected edge into a 0 bps expected edge if your position size is the full payout. Always haircut Kelly for settlement risk, and maintain a blacklist of markets with prior disputes.

1.9 Summary

This chapter developed the four arbitrage conditions in order of complexity: single-market YES+NO sum < $1 (riskless under maker execution); multi-outcome NegRisk sum drift (riskless in theory, concentrated in politics in practice); combinatorial covering portfolio via LP (riskless if all legs fill, the deepest edge but highest execution risk); cross-platform vs sportsbooks (directional with edge, settlement and capital risks). Each condition has a precise mathematical threshold that incorporates the current fee structure. The next chapter covers the data pipeline that feeds these detectors in real time.

CHAPTER 2 · DATA PIPELINE

Data & Market Discovery

An arbitrage scanner is only as good as its data feed. This chapter covers the Polymarket API stack end-to-end: the Gamma API for event and market metadata, the CLOB REST API for order book snapshots, and the CLOB WebSocket for real-time book deltas. We then cover how to group related markets by event, how to detect the logical dependencies between moneyline, spread, and totals markets that make combinatorial arbitrage possible, and how to model liquidity and slippage from the order book. Every code snippet in this chapter is taken directly from the production modules in the companion package.

2.1 The Polymarket API stack

Polymarket exposes three distinct API surfaces, each with a specific role. Understanding which API to use for which task is the first step in building a low-latency scanner.

API	Base URL	Transport	Use for
Gamma API	gamma-api.polymarket.com	HTTPS REST	Event metadata, market questions, slugs, tags, resolution criteria
CLOB REST	clob.polymarket.com	HTTPS REST	Order books, prices, last trades, order placement (auth)
CLOB WebSocket	wss://ws-subscriptions-clob.polymarket.com/ws	WebSocket	Real-time book deltas, last-trade prices, order fill notifications
Polygon RPC	polygon-rpc.com	JSON-RPC	On-chain reads (ERC-1155 balances, condition resolution)

Table 2.1 — Polymarket API surfaces and their roles

The Gamma API is the public metadata layer. It returns event listings, market questions, slugs, tags, and resolution criteria as JSON. It does not have order book data — that lives on the CLOB. The CLOB REST API is where prices live: it exposes /book?token_id=... for full order book snapshots, /price?token_id=...&side;=buy|sell|mid for top-of-book quotes, and authenticated endpoints for order placement. The CLOB WebSocket pushes real-time book deltas so you do not have to poll.

2.2 Fetching events and markets

The Gamma /events endpoint lists all active events with optional filters. Each event contains a list of markets. For sports arbitrage, we want to filter to sports-tagged events resolving within the next 12 hours (the daily sports cadence). Polymarket's tag filter is unreliable, so we apply client-side sport detection via regex on the event title and explicit tag labels.

Sport detection

The detect_sport() function in gamma_client.py checks the event's tags first (Polymarket sometimes tags Sports → NBA), then falls back to a regex match against the event title using team-name patterns. The regex dictionary covers the major NBA, NFL, MLB, NHL, soccer, tennis, and MMA team and league keywords. Events that match no pattern are classified as OTHER and filtered out of the sports scanner.

▸ **gamma_client.py — sport detection**

```
# From gamma_client.py — sport detection by regex fallback
_SPORT_PATTERNS: Dict[Sport, re.Pattern] = {
    Sport.NBA: re.compile(
        r"\b(NBA|Lakers|Celtics|Warriors|Nuggets|Bucks|Suns|Knicks|"
        r"76ers|Heat|Mavericks|Clippers|Kings|Hawks|Pacers|Bulls|"
        r"Cavaliers|Magic|Pistons|Raptors|Grizzlies|Pelicans|"
        r"Timberwolves|Spurs|Rockets|Thunder|Trail Blazers|Blazers|"
        r"Jazz|Nets)\b",
        re.IGNORECASE,
    ),
    Sport.NFL: re.compile(
        r"\b(NFL|Chiefs|Eagles|49ers|Bills|Cowboys|Ravens|Bengals|"
        r"Packers|Lions|Dolphins|Jets|Patriots|Steelers|Browns|"
        r"Texans|Colts|Jaguars|Titans|Broncos|Raiders|Chargers|"
        r"Seahawks|Rams|Cardinals|Falcons|Panthers|Saints|Buccaneers|"
        r"Vikings|Bears|Commanders|Giants)\b",
        re.IGNORECASE,
    ),
    # ... MLB, NHL, SOCCER, TENNIS, MMA ...
}

def detect_sport(title: str, tags: Optional[List[str]] = None) -&gt; Sport:
    """Detect sport from event title and tags."""
    if tags:
        for tag in tags:
            t = tag.lower()
            if "nba" in t: return Sport.NBA
            if "nfl" in t: return Sport.NFL
            # ... etc
    for sport, pattern in _SPORT_PATTERNS.items():
        if pattern.search(title):
            return sport
    return Sport.OTHER
```

Fetching events in parallel

The fetch_sports_events_with_markets() method lists sports events, then fetches full event details (including all markets) in parallel using asyncio.gather. This is critical for latency: a sequential fetch of 50 events at 200ms each would take 10 seconds, but a parallel fetch with a 16-connection pool completes in under a second.

▸ **gamma_client.py — parallel event fetching**

```
# From gamma_client.py — parallel event fetching
async def fetch_sports_events_with_markets(
    self,
    sports: Optional[List[Sport]] = None,
    max_events: int = 50,
) -&gt; List[Event]:
    raw_events = await self.list_sports_events(limit=max_events)
    if sports:
        sports_set = set(sports)
        raw_events = [
            ev for ev in raw_events
```

```
            if detect_sport(
                ev.get("title", ""),
                [t.get("label", "") for t in ev.get("tags", [])],
            ) in sports_set
        ]

    slugs = [ev.get("slug") for ev in raw_events if ev.get("slug")]
    detailed = await asyncio.gather(
        *[self.get_event(s) for s in slugs],
        return_exceptions=True,
    )

    events = []
    for raw in detailed:
        if isinstance(raw, Exception) or raw is None:
            continue
        try:
            events.append(self.parse_event(raw))
        except Exception as e:
            log.warning(f"Failed to parse event: {e}")
    return events
```

2.3 Logical dependency detection

Combinatorial arbitrage requires identifying which markets on the same event are logically dependent — that is, which markets share the same underlying outcome. For NBA games, this typically means the moneyline, the spread, the totals, and any player props on the same game. The detect_category() function classifies each market by regex-matching its question text against patterns for moneyline, spread, total, and player prop phrasings.

▸ **gamma_client.py — market category detection**

```
# From gamma_client.py — market category detection
_MONEYLINE_RE = re.compile(
    r"\b(will\s+win|moneyline|match\s+winner|game\s+winner)\b",
    re.IGNORECASE,
)
_SPREAD_RE = re.compile(
    r"([+-]?\d+\.?\d*)\s*(point|pt|run|puck|goal)\s*"
    r"(spread|line|handicap)|"
    r"\b(spread|handicap|line)\b\s*([+-]?\d+\.?\d*)",
    re.IGNORECASE,
)
_TOTAL_RE = re.compile(
    r"(over|under|o/u)\s*(\d+\.?\d*)|"
    r"(total|totals)\s*(over|under|o/u)?\s*(\d+\.?\d*)",
    re.IGNORECASE,
)
_PLAYER_PROP_RE = re.compile(
    r"\b(points|rebounds|assists|steals|blocks|threes|3-pointers|"
    r"passing\s+yards|rushing\s+yards|receiving\s+yards|"
    r"touchdowns|hits|home\s+runs|strikeouts|"
    r"goals|assists|saves)\b.*\b(over|under|o/u)\b",
    re.IGNORECASE,
)

def detect_category(question: str) -&gt; MarketCategory:
    if _PLAYER_PROP_RE.search(question):
        return MarketCategory.PLAYER_PROP
    if _SPREAD_RE.search(question):
        return MarketCategory.SPREAD
    if _TOTAL_RE.search(question):
```

```
        return MarketCategory.TOTAL
    if _MONEYLINE_RE.search(question):
        return MarketCategory.MONEYLINE
    return MarketCategory.OTHER
```

The parse_market() method also extracts structured fields: spread_value from spread questions (e.g., −3.5 from *Lakers −3.5*), total_value from totals questions (e.g., 220.5 from *Over 220.5*), and team_home / team_away for downstream feasibility constraints in the LP solver.

> **I NOTE**
> For markets that do not match any regex pattern (rare but possible for novel prop types), the scanner falls back to classifying them as OTHER and excluding them from combinatorial detection. A production system should periodically audit these OTHER markets and add new regex patterns as new market types appear on Polymarket.

2.4 Real-time data: polling vs WebSocket

Two transport modes are supported, chosen by scan context. For pre-game scanning (the bulk of opportunities), a 5-second polling interval on the CLOB REST API is sufficient because prices move slowly when no game is in progress. For in-play scanning (the highest-edge windows during live games), the WebSocket subscription is mandatory — sub-second price updates are required to catch the lag between a scoring play and the order book re-pricing.

Scan context	Poll interval	Transport	Rationale
Pre-game (idle)	5-10 sec	REST /book?token_id=	Lower API load, sufficient for slow-moving books
Pre-game (active)	2-3 sec	REST /book?token_id=	Tighter polling near tipoff when books active
In-play	Real-time push	WebSocket subscribe	Sub-second deltas, required for live edges
Post-game (resolution)	30 sec	REST /markets?id=	Watch for UMA dispute window

Table 2.2 — Polling strategy by scan context

WebSocket subscription and reconnection

The CLOBWSClient class in clob_client.py manages a persistent WebSocket connection with automatic reconnection. The subscribe message takes a list of token IDs and a type field ("market" for book updates, "last_trade_price" for trade ticks). On disconnect, the client reconnects with exponential backoff: 1s, 2s, 5s, 10s, 30s, 60s. A handler callback is invoked for every book update, allowing downstream detectors to react in real time.

▸ **clob_client.py — WebSocket client**

```
# From clob_client.py — WebSocket client with auto-reconnect
class CLOBWSClient:
    def __init__(self, cfg: Config):
        self.cfg = cfg
        self.url = cfg.api.ws_url
        self.ws: Optional[Any] = None
        self._books: Dict[str, OrderBook] = {}
```

```
        self._handlers: List[callable] = []
        self._running = False
        self._reconnect_attempts = 0

    def on_book_update(self, handler):
        """Register a callback: handler(token_id: str, book: OrderBook)."""
        self._handlers.append(handler)

    async def run(self, token_ids: List[str]):
        """Connect, subscribe, and pump messages until stopped."""
        import websockets
        self._running = True
        while self._running:
            try:
                async with websockets.connect(
                    self.url, ping_interval=20, ping_timeout=10,
                ) as ws:
                    self.ws = ws
                    self._reconnect_attempts = 0
                    await self.subscribe(token_ids)
                    async for raw_msg in ws:
                        if not self._running: break
                        await self._handle_message(raw_msg)
            except Exception as e:
                if not self._running: break
                self._reconnect_attempts += 1
                backoff = self._backoff()
                log.warning(
                    f"CLOB WS disconnected (attempt "
                    f"{self._reconnect_attempts}): {e}; reconnect in {backoff}s"
                )
                await asyncio.sleep(backoff)

    def _backoff(self) -&gt; float:
        backoffs = self.cfg.scanner.ws_reconnect_backoff
        idx = min(self._reconnect_attempts - 1, len(backoffs) - 1)
        return backoffs[idx]
```

2.5 Liquidity and depth analysis

Detecting an arbitrage edge is only half the battle — you also need to know how much you can actually fill without moving the market. The OrderBook dataclass in models.py provides several methods for depth-aware fill simulation.

Depth-weighted fill simulation

The simulate_buy_yes(shares) method walks the ask side of the book, consuming levels until the requested share count is filled. It returns the volume-weighted average price (VWAP) and the total cost. This is the same logic the detector uses to compute realistic slippage for a target notional.

▸ **models.py — depth simulation**

```
# From models.py — order book depth simulation
def simulate_buy_yes(self, shares: float) -&gt; Tuple[float, float]:
    """Simulate buying `shares` of YES at market (taker).
    Returns (avg_price, total_cost). Walks the ask side.
    """
    remaining = shares
    total_cost = 0.0
    for level in self.asks:
        take = min(remaining, level.size)
```

```
            total_cost += take * level.price
            remaining -= take
            if remaining &lt;= 1e-9:
                break
        filled = shares - remaining
        avg = total_cost / filled if filled &gt; 0 else 0.0
        return avg, total_cost

    def depth_usd(self, levels: int = 5) -&gt; Tuple[float, float]:
        """Return (bid_depth_usd, ask_depth_usd) for top N levels."""
        bid_depth = sum(l.price * l.size for l in self.bids[:levels])
        ask_depth = sum(l.price * l.size for l in self.asks[:levels])
        return bid_depth, ask_depth
```

Slippage estimation

The estimate_slippage_bps() function in risk.py computes the expected slippage in basis points for a target notional. It compares the VWAP from simulate_buy_yes() against the best ask, and returns the percentage difference. The risk gate rejects opportunities where estimated slippage exceeds cfg.risk.max_slippage_bps (default 50 bps).

▸ **risk.py — slippage estimation**

```
# From risk.py — slippage estimation
def estimate_slippage_bps(book, target_notional_usdc: float, cfg: Config) -&gt; float:
    """Estimate slippage in bps for a market order of given size."""
    if not book or not book.best_ask:
        return float("inf")
    shares_needed = target_notional_usdc / book.best_ask.price
    avg_price, _ = book.simulate_buy_yes(shares_needed)
    if avg_price &lt;= 0:
        return float("inf")
    slippage_fraction = (avg_price - book.best_ask.price) / book.best_ask.price
    return slippage_fraction * 10_000
```

2.6 Rate limiting and concurrency

Polymarket's CLOB API has a soft rate limit of approximately 10 requests per second per IP. The scanner respects this via two mechanisms: httpx.Limits caps concurrent connections, and a asyncio.Semaphore in get_books_for_markets() caps concurrent book fetches to 16 (configurable via cfg.api.max_concurrent_requests).

For higher-throughput scanning, you can shard by event slug and distribute across multiple VPS instances, each with its own IP. The Polygon RPC endpoint can also be load-balanced across Alchemy/Infura/QuickNode for on-chain reads. Chapter 4 covers the scaling architecture in detail.

2.7 Caching and snapshot persistence

Every scan cycle produces a snapshot of all market states at that instant. The backtest.py module's record_snapshots() function serializes these snapshots to JSON for later backtesting. Each snapshot contains the timestamp, the full event tree, and the top 10 levels of each market's order book. A typical 6-hour recording at 30-second intervals produces about 720 snapshots totaling 200-500 MB of JSON, depending on the number of active sports events.

For production monitoring, snapshots should also be written to a time-series database (TimescaleDB, InfluxDB, or ClickHouse) for queryable historical analysis. The companion code ships with JSON-only persistence for simplicity; production deployments should swap in a proper TSDB.

CHAPTER 3 · IMPLEMENTATION

Arbitrage Scanner: Full Implementation

This chapter walks through the complete production scanner, module by module. The accompanying Python package contains twelve runnable files totaling roughly 2,500 lines of code. Each section below explains the design decisions, presents the key code listings, and connects the module to the math developed in Chapter 1. By the end of this chapter you should be able to install the package, set your API credentials, and run the scanner in dry-run mode against live Polymarket data.

3.1 System architecture

The scanner is built as an async pipeline with five stages: data ingestion, opportunity detection, risk gating, execution, and position tracking. Each stage is a separate module with a clean interface, allowing individual components to be tested, replaced, or scaled independently.

Stage	Module(s)	Responsibility
1. Ingestion	gamma_client.py, clob_client.py	Fetch events, markets, order books (parallel)
2. Detection	detectors.py, optimizer.py	Run 4 arbitrage detectors per event
3. Risk gate	risk.py	Kelly sizing, exposure caps, blacklist, slippage check
4. Execution	executor.py	Concurrent leg placement, partial-fill hedging, rollback
5. Tracking	scanner.py (Scanner class)	Position state, PnL, alerts, persistence

Table 3.1 — Scanner pipeline stages

The pipeline runs in a single async event loop. Stage 1 fans out across events using asyncio.gather; stage 2 runs synchronously on the resolved data; stage 3 is a pure function with no I/O; stage 4 fans out across legs of an opportunity; stage 5 updates in-memory state. The loop sleeps cfg.scanner.poll_interval_sec seconds between cycles (default 5 seconds for sports).

3.2 Configuration module

All runtime knobs live in config.py as a hierarchy of dataclasses: ApiConfig, ScannerConfig, RiskConfig, ExecutionConfig, AlertsConfig. Configuration is loaded in three layers: code defaults, then environment variables (POLY_* prefix), then optional YAML file. This makes it easy to run the scanner with different configurations across environments (dev, staging, production) without code changes.

▸ **config.py — configuration dataclasses**

```
# From config.py — configuration dataclasses
@dataclass
class ScannerConfig:
    """Opportunity detection thresholds."""
    min_edge_taker_bps: int = 75          # must clear 0.75% taker fee
    min_edge_maker_bps: int = 30          # maker fee = 0, just slippage buffer
    min_edge_negrisk_bps: int = 100       # NegRisk rebalance needs higher threshold
    min_liquidity_usdc: float = 5_000     # each side must have ≥ this depth
    min_depth_levels: int = 3
```

```
    sports_only: bool = True
    resolve_within_hours: int = 12        # daily sports cadence
    poll_interval_sec: float = 5.0
    in_play_poll_interval_sec: float = 1.0
    ws_reconnect_backoff: List[float] = field(
        default_factory=lambda: [1, 2, 5, 10, 30, 60]
    )

@dataclass
class RiskConfig:
    """Position sizing and exposure limits."""
    max_position_usdc: float = 5_000
    max_game_exposure_usdc: float = 15_000
    max_daily_exposure_usdc: float = 100_000
    max_open_positions: int = 25
    kelly_fraction: float = 0.25          # 25% of full Kelly
    kelly_floor_bps: int = 10
    kelly_cap_bps: int = 500              # never more than 5% of bankroll per leg
    bankroll_usdc: float = 50_000
    slippage_bps_per_1k_usdc: float = 5.0
    max_slippage_bps: int = 50
```

The fee constants are defined at module level for easy reference: TAKER_FEE_BPS = 75 and MAKER_FEE_BPS = 0. Helper functions taker_fee_rate() and maker_fee_rate() return these as fractions (0.0075 and 0.0) for use in detector math.

3.3 Domain models

The models.py module defines the core dataclasses: Market, OrderBook, Event, Opportunity, Leg, Position, and AtomicOutcome. These are pure data containers with no I/O — they exist to give the rest of the codebase a shared vocabulary.

▸ **models.py — core domain classes**

```
# From models.py — core domain classes
@dataclass
class OrderBookLevel:
    price: float    # 0..1, in dollars
    size: float     # shares available at this level

@dataclass
class OrderBook:
    bids: List[OrderBookLevel] = field(default_factory=list)
    asks: List[OrderBookLevel] = field(default_factory=list)
    timestamp: float = 0.0

    @property
    def best_bid(self) -&gt; Optional[OrderBookLevel]:
        return self.bids[0] if self.bids else None

    @property
    def best_ask(self) -&gt; Optional[OrderBookLevel]:
        return self.asks[0] if self.asks else None

    @property
    def mid(self) -&gt; Optional[float]:
        if self.best_bid and self.best_ask:
            return (self.best_bid.price + self.best_ask.price) / 2
        return None

    @property
    def spread(self) -&gt; Optional[float]:
```

```
        if self.best_bid and self.best_ask:
            return self.best_ask.price - self.best_bid.price
        return None

@dataclass
class Market:
    condition_id: str
    question: str
    slug: str
    event_slug: str
    token_id_yes: str
    token_id_no: str
    market_type: MarketType
    sport: Sport = Sport.OTHER
    category: MarketCategory = MarketCategory.OTHER
    book: Optional[OrderBook] = None
    spread_value: Optional[float] = None
    total_value: Optional[float] = None
    # ... additional fields

    @property
    def price_yes(self) -&gt; Optional[float]:
        if self.book and self.book.best_ask:
            return self.book.best_ask.price
        return None

    @property
    def price_no(self) -&gt; Optional[float]:
        if self.book and self.book.best_bid:
            return 1.0 - self.book.best_bid.price
        return None
```

3.4 Single-market YES+NO detector

The detect_single_market_arb() function in detectors.py is the bread-and-butter detector. It implements the math from Section 1.3: compute YES ask + NO ask, check if the sum is below $1 after fees, and if so construct an Opportunity object with both legs.

▸ **detectors.py — single-market detector**

```
# From detectors.py — single-market arb detector (simplified)
def detect_single_market_arb(market: Market, cfg: Config) -&gt; Optional[Opportunity]:
    if market.book is None or market.market_type != MarketType.BINARY:
        return None
    book = market.book
    if not book.best_ask or not book.best_bid:
        return None
    if not _liquidity_check(book, cfg.scanner.min_liquidity_usdc):
        return None

    # Taker path: cross both sides
    yes_ask = book.best_ask.price
    no_cost = 1.0 - book.best_bid.price  # buy NO = sell YES at bid
    total_cost_taker = yes_ask + no_cost
    payout_per_share = 1.0
    fee_taker = total_cost_taker * taker_fee_rate()
    net_cost_taker = total_cost_taker + fee_taker
    net_edge_taker = payout_per_share - net_cost_taker
    edge_bps_taker = _bps(net_edge_taker / total_cost_taker)

    # Maker path: rest both sides at mid ± spread/2
    if book.mid is not None:
```

```
        spread = book.spread or 0
        maker_total_cost = 1.0 - spread
        maker_edge = spread
        edge_bps_maker = _bps(maker_edge / maker_total_cost)
    else:
        edge_bps_maker = 0

    # Pick better path
    best_edge_bps = max(edge_bps_taker, edge_bps_maker)
    is_maker = edge_bps_maker &gt; edge_bps_taker
    threshold = (cfg.scanner.min_edge_maker_bps if is_maker
                 else cfg.scanner.min_edge_taker_bps)
    if best_edge_bps &lt; threshold:
        return None

    # Size by min of bid/ask depth and position cap
    bid_depth, ask_depth = book.depth_usd(levels=5)
    max_shares_by_depth = min(
        ask_depth / yes_ask if yes_ask &gt; 0 else 0,
        bid_depth / no_cost if no_cost &gt; 0 else 0,
    )
    cost_per_share = total_cost_taker if not is_maker else maker_total_cost
    max_shares_by_position = cfg.risk.max_position_usdc / cost_per_share
    shares = min(max_shares_by_depth, max_shares_by_position)
    if shares &lt; 10:  # dust filter
        return None

    # Build legs and Opportunity
    legs = [
        Leg(market=market, side=Side.YES, price=yes_ask,
            shares=shares, cost=yes_ask * shares, is_maker=is_maker),
        Leg(market=market, side=Side.NO, price=no_cost,
            shares=shares, cost=no_cost * shares, is_maker=is_maker),
    ]
    total_cost = sum(l.cost for l in legs)
    net_cost = total_cost if is_maker else total_cost * (1 + taker_fee_rate())
    guaranteed_payout = shares
    return Opportunity(
        id=_opp_id(market.condition_id, int(time.time() * 1000)),
        type=OpportunityType.SINGLE_MARKET,
        event_slug=market.event_slug,
        legs=legs, total_cost=total_cost,
        guaranteed_payout=guaranteed_payout,
        gross_edge=guaranteed_payout - total_cost,
        net_edge=guaranteed_payout - net_cost,
        net_edge_bps=_bps((guaranteed_payout - net_cost) / total_cost),
        detected_at=time.time(),
        expires_at=time.time() + 30,
    )
```

3.5 Multi-outcome NegRisk detector

The detect_negrisk_rebalance_arb() function implements the math from Section 1.4. It iterates over all NegRisk markets on an event, computes the sum of YES asks and the sum of YES bids, then checks both the buy-all-YES and buy-all-NO conditions. Per the article framing, this detector is mostly relevant for politics — sports NegRisk markets are rare and thin — but it is included for completeness and for the occasional MVP or championship winner market that does appear in sports.

▸ **detectors.py — NegRisk rebalance detector**

```
# From detectors.py — NegRisk rebalance detector (simplified)
```

```
def detect_negrisk_rebalance_arb(event: Event, cfg: Config) -&gt; Optional[Opportunity]:
    neg_markets = [m for m in event.markets if m.neg_risk and m.book]
    if len(neg_markets) &lt; 3:
        return None

    sum_yes_asks = 0.0
    sum_yes_bids = 0.0
    valid_markets = []
    for m in neg_markets:
        if not m.book or not m.book.best_ask or not m.book.best_bid:
            continue
        sum_yes_asks += m.book.best_ask.price
        sum_yes_bids += m.book.best_bid.price
        valid_markets.append(m)

    if len(valid_markets) &lt; 3:
        return None

    # Buy-all-YES arb: sum(asks) &lt; $1
    fee_taker = sum_yes_asks * taker_fee_rate()
    net_cost_buy_all = sum_yes_asks + fee_taker
    edge_buy_all = 1.0 - net_cost_buy_all
    edge_bps_buy = _bps(edge_buy_all / sum_yes_asks)

    # Buy-all-NO arb: sum(bids) &gt; $1
    n = len(valid_markets)
    no_total_cost = n - sum_yes_bids
    no_fee = no_total_cost * taker_fee_rate()
    no_payout = n - 1
    edge_sell_all = no_payout - (no_total_cost + no_fee)
    edge_bps_sell = _bps(edge_sell_all / no_total_cost)

    best_edge_bps = max(edge_bps_buy, edge_bps_sell)
    if best_edge_bps &lt; cfg.scanner.min_edge_negrisk_bps:
        return None

    # Construct legs for whichever path won...
    # (see full implementation in detectors.py)
    return opportunity
```

3.6 Combinatorial detector and LP solver

The combinatorial detector is the most sophisticated piece of the scanner. It enumerates all feasible atomic outcomes for an event, builds a payoff matrix, and solves the covering portfolio LP using PuLP (an open-source MILP solver). The solve_covering_lp() function in optimizer.py implements the LP formulation from Section 1.5.

Feasibility pruning

The is_feasible_outcome() function prunes physically impossible atomic outcomes before passing them to the LP solver. For example, in an NBA game where the home team is favored by 3.5 points, the combination *away team wins AND home team covers −3.5* is impossible (if the away team wins, the home team cannot have covered a negative spread). Pruning these impossible combinations is critical: without it, the LP would waste effort covering outcomes that can never occur and might report a false arbitrage.

▸ **optimizer.py — feasibility pruning**

```
# From optimizer.py — feasibility pruning
def is_feasible_outcome(outcome: AtomicOutcome, markets: List[Market]) -&gt; bool:
```

```
    """Check whether an atomic outcome is physically possible."""
    moneylines = [m for m in markets if m.category == MarketCategory.MONEYLINE]
    spreads = [m for m in markets if m.category == MarketCategory.SPREAD]

    # Find moneyline winner
    ml_winner = None
    for m in moneylines:
        if m.condition_id in outcome.outcomes:
            res = outcome.outcomes[m.condition_id]
            ml_winner = "home" if res == "YES" else "away"

    # Check spread consistency
    for m in spreads:
        if m.condition_id not in outcome.outcomes:
            continue
        spread_res = outcome.outcomes[m.condition_id]
        if m.spread_value is not None:
            home_favored = m.spread_value &lt; 0
            home_covers = spread_res == "YES"
            # If home favored and away wins, home cannot cover
            if home_favored and ml_winner == "away" and home_covers:
                return False
            # If home underdog and home wins, home must cover
            if not home_favored and ml_winner == "home" and not home_covers:
                return False
    return True
```

LP solver

The LP itself is straightforward PuLP: minimize total cost subject to coverage constraints. The solver is called with a 5-second time limit; if it does not converge in that window, the detector skips the event. For very large events (many markets, many outcomes), the LP can be slow — Chapter 4 covers scaling strategies including sharding by market category.

▸ **optimizer.py — LP solver**

```
# From optimizer.py — LP solver
def solve_covering_lp(markets, atomic_outcomes, cfg):
    if not HAS_PULP:
        return None
    feasible = [o for o in atomic_outcomes if is_feasible_outcome(o, markets)]
    if not feasible:
        return None

    # Candidate positions: (YES, NO) for each market
    positions = []
    for m in markets:
        if not m.book: continue
        if m.book.best_ask: positions.append((m, Side.YES))
        if m.book.best_bid: positions.append((m, Side.NO))

    prob = pulp.LpProblem("covering_portfolio", pulp.LpMinimize)
    x = {
        (m.condition_id, s): pulp.LpVariable(
            f"x_{m.condition_id[:8]}_{s.value}", lowBound=0
        )
        for (m, s) in positions
    }

    # Objective: minimize Σ cost * x
    prob += pulp.lpSum(
        _position_cost((m, s)) * x[(m.condition_id, s)]
```

```
        for (m, s) in positions
    )

    # Constraints: each atomic outcome covered
    for j, outcome in enumerate(feasible):
        prob += pulp.lpSum(
            (1.0 if _position_pays((m, s), outcome) else 0.0)
            * x[(m.condition_id, s)]
            for (m, s) in positions
        ) &gt;= 1.0

    prob.solve(pulp.PULP_CBC_CMD(msg=0, timeLimit=5))
    if prob.status != pulp.constants.LpStatusOptimal:
        return None

    allocation = {}
    for (m, s) in positions:
        val = x[(m.condition_id, s)].value() or 0.0
        if val &gt; 1e-6:
            allocation[(m, s)] = val
    return pulp.value(prob.objective), allocation
```

Greedy fallback

If PuLP is not installed or the LP fails, the greedy_cover_heuristic() function provides a set-cover approximation with O(log N) ratio from optimal. It is useful as a fast pre-filter: if the greedy heuristic finds no covering portfolio below $1, the LP almost certainly will not either, so you can skip the LP call entirely on most events.

3.7 Risk module: Kelly sizing and exposure caps

The risk.py module implements the conservative Kelly sizing formula from Section 1.8, the per-event and per-day exposure caps, the persistent blacklist, and the slippage estimator. The risk gate is the final pre-execution checkpoint: if any constraint is violated, the opportunity is rejected or downsized.

Conservative Kelly

Full Kelly sizing for a guaranteed-win bet would suggest betting 100% of bankroll, which is clearly wrong in practice because of settlement risk, leg-failure risk, and execution variance. The scanner applies two haircuts: a perceived win probability of 0.95 (instead of 1.0) to account for settlement risk, and a configurable Kelly fraction (default 0.25, i.e., quarter-Kelly) to dampen variance further. The result is a position size typically 5-15% of full Kelly, which is the standard conservative regime for arbitrage strategies.

▸ **risk.py — Kelly sizing**

```
# From risk.py — conservative Kelly sizing
def kelly_fraction(p_win: float, odds_decimal: float) -&gt; float:
    """Full Kelly fraction of bankroll to bet."""
    if p_win &lt;= 0 or p_win &gt;= 1:
        return 0.0
    q = 1 - p_win
    b = odds_decimal - 1
    if b &lt;= 0:
        return 0.0
    f = (p_win * b - q) / b
    return max(0.0, f)

def conservative_kelly_size(
    edge_fraction: float, bankroll: float, cfg: Config,
    perceived_p_win: float = 0.95,
```

```
) -&gt; float:
    """Conservative Kelly sizing for an arb opportunity."""
    if edge_fraction &lt;= 0:
        return 0.0
    odds_decimal = 1.0 + edge_fraction
    f_full = kelly_fraction(perceived_p_win, odds_decimal)
    f_adjusted = f_full * cfg.risk.kelly_fraction
    max_bet = bankroll * cfg.risk.kelly_cap_bps / 10_000
    min_bet = bankroll * cfg.risk.kelly_floor_bps / 10_000
    bet = bankroll * f_adjusted
    bet = min(bet, max_bet)
    if bet &lt; min_bet:
        return 0.0
    return bet
```

Risk gate

The risk_gate() function runs all pre-execution checks in order. If any check fails, it returns a RiskCheckResult with passed=False and the rejection reason. If a check would be violated by the full position size but a smaller size would pass, it returns passed=True with adjusted_size_usdc set to the maximum allowed size.

▸ **risk.py — risk gate**

```
# From risk.py — risk gate
def risk_gate(opportunity, tracker, blacklist, cfg) -&gt; RiskCheckResult:
    # 1. Blacklist check
    for leg in opportunity.legs:
        if blacklist.contains(leg.market.condition_id):
            return RiskCheckResult(passed=False, reason="blacklisted market")

    # 2. Per-event exposure cap
    event_exp = tracker.event_exposure(opportunity.event_slug)
    if event_exp + opportunity.total_cost &gt; cfg.risk.max_game_exposure_usdc:
        remaining = cfg.risk.max_game_exposure_usdc - event_exp
        if remaining &lt; 100:
            return RiskCheckResult(passed=False, reason="per-event cap reached")
        scale = remaining / opportunity.total_cost
        return RiskCheckResult(passed=True,
            adjusted_size_usdc=opportunity.total_cost * scale,
            reason=f"downsized to fit per-event cap")

    # 3. Daily cap, 4. open positions cap, 5. per-position cap...
    # 6. Slippage check
    sample_leg = opportunity.legs[0]
    if sample_leg.market.book:
        slip_bps = estimate_slippage_bps(
            sample_leg.market.book, opportunity.total_cost, cfg
        )
        if slip_bps &gt; cfg.risk.max_slippage_bps:
            return RiskCheckResult(passed=False, reason="slippage exceeds cap")

    # 7. Kelly sizing
    edge_fraction = opportunity.net_edge / opportunity.total_cost
    kelly_size = conservative_kelly_size(
        edge_fraction=edge_fraction,
        bankroll=cfg.risk.bankroll_usdc,
        cfg=cfg,
    )
    if kelly_size &lt; opportunity.total_cost:
        return RiskCheckResult(passed=True,
            adjusted_size_usdc=kelly_size,
            reason=f"Kelly downsized to ${kelly_size:.0f}")
    return RiskCheckResult(passed=True)
```

3.8 Execution engine

The Executor class in executor.py places all legs of an opportunity, with concurrent placement when cfg.execution.concurrent_legs is true (default). Each leg tries maker execution first (limit order at the desired price), falling back to taker (market order) if the maker order does not fill within cfg.execution.maker_timeout_sec seconds.

Concurrent leg placement

Concurrent placement is critical for arbitrage: if you place legs sequentially, the price can move between legs and you end up with unhedged exposure. The scanner uses asyncio.gather to submit all leg orders simultaneously, then waits for all to complete before assessing fill results.

▸ **executor.py — concurrent leg placement**

```
# From executor.py — concurrent leg placement
async def execute_opportunity(self, opportunity, adjusted_size_usdc=None):
    legs = opportunity.legs
    if adjusted_size_usdc and adjusted_size_usdc &lt; opportunity.total_cost:
        scale = adjusted_size_usdc / opportunity.total_cost
        legs = [Leg(market=l.market, side=l.side, price=l.price,
                    shares=l.shares * scale, cost=l.cost * scale,
                    is_maker=l.is_maker) for l in opportunity.legs]

    if self._dry_run:
        return self._dry_run_position(opportunity, legs), [...]

    # Execute legs concurrently
    if self.cfg.execution.concurrent_legs:
        results = await asyncio.gather(*[self._execute_leg(l) for l in legs])
    else:
        results = [await self._execute_leg(l) for l in legs]

    # Hedge any partial fills
    results = await self._hedge_partials(opportunity, list(results))

    position = self._build_position(opportunity, legs, results)
    return position, results
```

Partial-fill hedging

If some legs fill completely and others fill partially, you are left with unhedged exposure on the unfilled portion. The _hedge_partials() method computes the minimum filled share count across all legs, then sells the excess on any leg that over-filled. This locks in the matched portion as a complete arb and leaves you with no unhedged directional risk.

▸ **executor.py — partial-fill hedging**

```
# From executor.py — partial-fill hedging
async def _hedge_partials(self, opportunity, results):
    if not self.cfg.execution.hedge_on_partial_fill:
        return results
    has_partial = any(r.is_partial or r.status == OrderStatus.FAILED
                      for r in results)
    if not has_partial:
        return results

    # Min filled across all legs = matched amount
```

```
    min_filled = min(
        (r.filled_shares for r in results if r.filled_shares &gt; 0),
        default=0,
    )

    # Sell excess on each over-filled leg
    hedged = []
    for r in results:
        if r.filled_shares &gt; min_filled * 1.01:
            excess = r.filled_shares - min_filled
            token_id = (r.leg.market.token_id_yes
                        if r.leg.side == Side.YES
                        else r.leg.market.token_id_no)
            try:
                await self.order_client.place_market_order(
                    token_id=token_id, side="SELL", size=excess,
                )
                r.filled_shares = min_filled
            except Exception as e:
                log.warning(f"Hedge failed: {e}")
        hedged.append(r)
    return hedged
```

Rollback on critical failure

If execution fails catastrophically (e.g., 2 of 4 legs failed and the position is now unhedged directional risk), the rollback_position() function attempts to flatten the position by selling all filled shares at market. This is the last line of defense against leg-failure risk.

3.9 Backtesting framework

The backtest.py module provides a framework for testing the scanner against historical snapshots. It loads snapshots from JSON files, replays them through the detector stack, simulates execution with realistic slippage and fees, and reports performance metrics including total PnL, ROI, win rate, average edge, max drawdown, and per-strategy-type breakdowns.

▸ **backtest.py — backtest driver**

```
# From backtest.py — backtest driver (simplified)
def run_backtest(snapshots, cfg, bankroll_usdc=50_000, sports_filter=None):
    result = BacktestResult()
    result.n_snapshots = len(snapshots)

    for snap in snapshots:
        for event in snap.events:
            if sports_filter and event.sport not in sports_filter:
                continue
            opps = scan_event_for_opportunities(event, cfg)

            for opp in opps:
                result.n_opportunities += 1
                # Simulate execution with slippage
                total_filled_cost = 0.0
                all_filled = True
                for leg in opp.legs:
                    sim = simulate_fill(
                        book=leg.market.book, side="BUY",
                        target_shares=leg.shares, is_maker=leg.is_maker,
                    )
                    if sim.filled_shares &lt; leg.shares * 0.95:
```

```
                        all_filled = False; break
                    total_filled_cost += sim.filled_shares * sim.avg_fill_price

                if not all_filled:
                    result.n_failed += 1; continue

                # Compute PnL with fees
                fee = total_filled_cost * (
                    maker_fee_rate() if all(l.is_maker for l in opp.legs)
                    else taker_fee_rate()
                )
                net_cost = total_filled_cost + fee
                payout = opp.guaranteed_payout
                realized = payout - net_cost

                result.realized_pnl_usdc += realized
                result.total_cost_usdc += net_cost
                result.edge_history_bps.append(opp.net_edge_bps)
                # ... track by type, drawdown, etc.

    return result
```

3.10 Alerts module

The alerts.py module sends notifications to Telegram and Discord when opportunities are detected, orders fill, or errors occur. Alerts are filtered by edge threshold (default: only alert on ≥100 bps edges) to avoid notification fatigue. The Telegram transport uses the Telegram Bot API; the Discord transport uses incoming webhooks.

▸ **alerts.py — opportunity alert**

```
# From alerts.py — opportunity alert (Telegram + Discord)
async def send_opportunity_alert(self, opp: Opportunity):
    if opp.net_edge_bps &lt; self.cfg.min_edge_alert_bps:
        return
    msg = (
        f"[ARB] *Arb Opportunity*\n"
        f"Type: `{opp.type.value}`\n"
        f"Event: `{opp.event_slug}`\n"
        f"Net edge: *{opp.net_edge_bps:.0f} bps*\n"
        f"Cost: ${opp.total_cost:.2f}\n"
        f"Guaranteed payout: ${opp.guaranteed_payout:.2f}\n"
        f"Net profit: ${opp.net_edge:.2f}\n"
        f"Legs: {len(opp.legs)}"
    )
    await self._send_all(msg)

async def _send_telegram(self, text: str):
    if not self.cfg.telegram_bot_token or not self.cfg.telegram_chat_id:
        return
    url = f"https://api.telegram.org/bot{self.cfg.telegram_bot_token}/sendMessage"
    payload = {
        "chat_id": self.cfg.telegram_chat_id,
        "text": text, "parse_mode": "Markdown",
        "disable_web_page_preview": True,
    }
    try:
        await self.client.post(url, json=payload)
    except Exception as e:
        log.warning(f"Telegram alert error: {e}")
```

3.11 Main scanner loop

The Scanner class in scanner.py ties everything together. The run_forever() method is the main loop: fetch events, fetch order books, run detectors, filter through risk gate, execute, alert, sleep, repeat. Graceful shutdown is handled via SIGINT/SIGTERM signal handlers.

▸ scanner.py — main loop

```
# From scanner.py — main loop (simplified)
async def run_forever(self):
    self._running = True
    def _signal_handler(sig, frame):
        self._running = False
    signal.signal(signal.SIGINT, _signal_handler)
    signal.signal(signal.SIGTERM, _signal_handler)

    while self._running:
        try:
            opps = await self.scan_once()
            for opp in opps:
                if not self._running: break
                await self.alerts.send_opportunity_alert(opp)

                # Risk gate
                check = risk_gate(opp, self.tracker, self.blacklist, self.cfg)
                if not check.passed:
                    log.info(f"Skipping opp {opp.id}: {check.reason}")
                    continue

                # Execute
                position, results = await self.executor.execute_opportunity(
                    opp, adjusted_size_usdc=check.adjusted_size_usdc,
                )
                if position.status.value == "FILLED":
                    self.tracker.add(opp)
                await self.alerts.send_fill_alert(position)

            await asyncio.sleep(self.cfg.scanner.poll_interval_sec)
        except Exception as e:
            log.error(f"Scan cycle error: {e}", exc_info=True)
            await self.alerts.send_error_alert(str(e), context="scan_cycle")
            await asyncio.sleep(5.0)
    await self.stop()
```

3.12 CLI entry point

The main.py module exposes a CLI with four subcommands: scan (run live scanner), backtest (run backtest on saved snapshots), record (record live snapshots), and test-api (quick connectivity test). All commands accept --config for YAML config and --sports for sport filter.

▸ main.py test-api output (example)

```
$ python main.py test-api
Testing Polymarket API connectivity...

✓ Gamma API: fetched 10 sports events
    - Lakers vs Celtics (lakers-vs-celtics-jan-15)
    - ...

✓ CLOB API: fetched book for abc123def456
    Question: Will the Lakers defeat the Celtics on January 15?
```

```
      Best bid: 0.51
      Best ask: 0.52
      Spread:   0.0100
      Bid depth (5 levels): $4,237.50
      Ask depth (5 levels): $3,891.20

  Taker fee: 75 bps (0.75%)
  Maker fee: 0 bps (0.00%)
```

3.13 Installation and first run

The companion package ships with a requirements.txt and a README.md. Installation is standard Python:

▸ **Installation and first run**

```
$ cd arb_scanner/
$ python -m venv venv &amp;amp;&amp;amp; source venv/bin/activate
$ pip install -r requirements.txt

# Test API connectivity (no auth needed)
$ python main.py test-api

# Show current config (defaults + env vars)
$ python main.py show-config

# Run one scan cycle in dry-run mode
$ python main.py scan --once --sports NBA

# Run continuous scanner in dry-run
$ python main.py scan --sports NBA

# Go live (requires API keys)
$ python main.py scan --live --sports NBA
```

Before going live, set the required environment variables: POLY_API_KEY, POLY_API_SECRET, POLY_API_PASSPHRASE, POLY_PRIVATE_KEY, and POLY_WALLET_ADDRESS. Optional variables include POLY_TG_TOKEN and POLY_TG_CHAT_ID for Telegram alerts, POLY_DISCORD_WEBHOOK for Discord, and POLY_BANKROLL_USDC to override the default bankroll. Never commit real keys to version control.

CHAPTER 4 · PRODUCTION HARDENING

Production Hardening

A scanner that works in dry-run is not a scanner that prints money. This chapter covers the operational engineering required to take the implementation from Chapter 3 and run it profitably in production: low-latency infrastructure, slippage and fee modeling, in-play execution, monitoring, scaling architecture, and compliance. Every recommendation here comes from production incidents — either experienced directly or observed in the broader Polymarket operator community.

4.1 Low-latency infrastructure

Latency in the scanner pipeline has three components: data ingestion (fetching events and books), detection (running the detectors on the resolved data), and execution (placing orders). For pre-game sports arbitrage, total latency budgets of 1-2 seconds are acceptable because edges persist for tens of seconds. For in-play arbitrage, the budget collapses to 200-500 milliseconds — the edge window after a scoring play is often under 5 seconds, and competing bots will close the gap within 2 seconds.

VPS region selection

Polymarket's CLOB API is hosted on AWS us-east-1. The Polygon RPC endpoints are geographically distributed, but the canonical polygon-rpc.com endpoint is also us-east-1. Running the scanner from a us-east-1 VPS gives single-digit-millisecond API latency. Running from Europe or Asia adds 80-200ms of network round-trip, which is acceptable for pre-game but marginal for in-play. For serious in-play operation, a us-east-1 VPS (or co-located bare metal) is essentially mandatory.

VPS region	Latency to CLOB	Cost	Notes
AWS us-east-1 (Virginia)	1-5 ms	$5-50/mo	Best for in-play; co-located with Polymarket infra
AWS us-west-2 (Oregon)	60-80 ms	$5-50/mo	Acceptable for pre-game only
GCP us-central1 (Iowa)	20-40 ms	$5-50/mo	Good compromise
EU (Frankfurt)	80-100 ms	€5-50/mo	Marginal even for pre-game
Asia (Singapore)	200-250 ms	$5-50/mo	Not recommended

Table 4.1 — VPS region latency to Polymarket CLOB API

WebSocket reconnection

WebSocket disconnects are inevitable. The CLOBWSClient in clob_client.py handles reconnection with exponential backoff (1s, 2s, 5s, 10s, 30s, 60s), but you should also implement a heartbeat monitor that alerts if the WebSocket has been disconnected for more than 60 seconds. A silent disconnection is the most common cause of stale-data trades: the scanner thinks it has fresh books but is actually trading against a snapshot from minutes ago.

For high-availability deployment, run two scanner instances in different us-east-1 availability zones (us-east-1a and us-east-1b). The primary instance executes trades; the secondary watches the same data stream and alerts if it

detects an opportunity the primary missed. This catches both instance-level failures and order-placement bugs.

Rate limit handling

Polymarket's CLOB API has a soft rate limit of approximately 10 requests per second per IP. The scanner respects this via httpx.Limits (max connections) and a asyncio.Semaphore in get_books_for_markets() (max concurrent book fetches). If you exceed the limit, the API returns HTTP 429 with a Retry-After header. The scanner's retry logic in _get() handles this automatically.

For higher throughput, shard by event slug across multiple VPS instances, each with its own IP. A two-instance shard can double effective throughput; a four-instance shard can quadruple it. Coordinate via a shared Redis instance to avoid duplicate executions of the same opportunity.

4.2 Slippage and fee modeling

Theoretical edge is what the detector reports; realized edge is what hits your bankroll after slippage and fees. The gap between the two is the single biggest source of disappointment for new arbitrage operators. This section covers the slippage and fee model used by the scanner's risk gate.

Depth-weighted slippage

The estimate_slippage_bps() function walks the order book to compute the volume-weighted average price (VWAP) for a target notional, then compares it to the best ask. For small orders in liquid books, slippage is near zero. For large orders in thin books, slippage can easily exceed the arbitrage edge itself. The risk gate rejects opportunities where estimated slippage exceeds cfg.risk.max_slippage_bps (default 50 bps).

A common mistake is to model slippage linearly — for example, 5 bps per $1,000 of notional. In reality, slippage is non-linear: it stays near zero until your order starts consuming levels beyond the top of book, then rises sharply. The scanner's book-walking simulator captures this non-linearity accurately. Always trust the simulator over a linear approximation.

Fee-aware threshold adjustment

The min-edge thresholds in ScannerConfig are split by execution mode: min_edge_taker_bps = 75 (must clear the 0.75% taker fee exactly) and min_edge_maker_bps = 30 (just slippage buffer, since maker fee is zero). For 50/50 markets where the taker fee bites hardest, consider raising the taker threshold to 100 bps to leave a profit margin after slippage. For deep markets with tight spreads, the maker threshold can drop to 15-20 bps.

Liquidity regime	Fill probability	Break-even sum	Maker threshold	Taker threshold
Deep (>$10k each side)	0.97	0.9926	15-25 bps	75-100 bps
Medium ($1k-$10k)	0.95	0.9926	25-40 bps	100-150 bps
Thin (<$1k)	0.90	0.9926	50+ bps	Skip (slippage too high)
In-play (live game)	0.85	0.9926	30-50 bps	150+ bps

Table 4.2 — Recommended edge thresholds by liquidity regime

4.3 In-play / live opportunities

In-play arbitrage — trading during live games — is where the highest edges live, but it is also the most operationally demanding. The edge windows open and close in seconds, and competing bots are watching the same data feed. The good news: sports retail flow is at its most emotional during live games, which means prices are at their most inefficient.

When in-play edges appear

The three highest-edge windows during a live game are: (1) immediately after a scoring play, when the order book lags the new game state by 1-3 seconds; (2) after an injury announcement, when the market re-prices over 5-15 seconds; (3) during momentum shifts (e.g., a 10-0 run in basketball), when emotional betting creates transient mispricings. Each window has a different edge profile and requires a different execution strategy.

Faster execution path for in-play

The standard scanner loop (5-second poll interval, full event refresh) is too slow for in-play. The in-play execution path subscribes to the CLOB WebSocket for the specific token IDs of in-progress games, runs the single-market detector on every book delta, and executes via market orders (taker) rather than limit orders (maker). The maker path is too slow for in-play because limit orders may not fill before the edge closes.

The trade-off: in-play execution pays the 0.75% taker fee on both legs, so the gross edge must exceed 150 bps to be profitable. This is a much higher bar than pre-game maker execution, but the in-play edge windows are also much larger — a 500-1000 bps edge during a momentum shift is not uncommon.

> **! WARNING**
> In-play execution is the highest-risk mode. Books move fast, partial fills are common, and a single missed leg can leave you with significant unhedged directional exposure. Run in-play with reduced position sizes (50% of pre-game Kelly) and a tighter slippage cap (25 bps max). Never run in-play unattended until you have logged at least 100 hours of dry-run observation.

4.4 Monitoring dashboard

A scanner without monitoring is a scanner that will quietly bleed money when something goes wrong. The minimum viable monitoring stack tracks PnL, edge history, fill rates, and slippage distribution. For production, add leader-board correlation (tracking the wallets of known profitable operators) and per-strategy Sharpe ratios.

Key metrics to track

- **Realized PnL** — running total, daily, weekly, monthly. Plot as time series; alert on drawdown exceeding 2× expected daily volatility.
- **Edge history** — histogram of detected edge bps. Should be roughly log-normal with a long right tail. If the distribution shifts left, the market is becoming more efficient and your thresholds need adjustment.
- **Fill rate** — fraction of attempted legs that filled completely. Pre-game maker execution typically fills 60-80%; in-play taker execution should fill 90%+. Drops below 50% indicate a stale book or a broken order-placement path.

- **Slippage distribution** — actual slippage in bps, comparing avg fill price to detector-reported price. If actual slippage consistently exceeds estimated slippage, the slippage model needs recalibration.
- **Per-strategy Sharpe** — rolling 30-day Sharpe ratio by strategy type. Kill switches on any strategy whose Sharpe drops below 1.0 (annualized).
- **Leaderboard correlation** — track the top 10 Polymarket wallets by PnL. If your fills start correlating with theirs, you are competing for the same edges; if your fills diverge from theirs, either you found a new edge or you are missing the existing one.

Dashboard implementation

For a lightweight dashboard, use Streamlit or Grafana backed by a TimescaleDB or ClickHouse instance. The scanner writes a row to the database for every opportunity detected and every position closed, with columns for timestamp, strategy type, edge, cost, payout, realized PnL, slippage, and fill rate. Dashboards query this table directly with time-window aggregations.

4.5 Scaling architecture

A single scanner instance can comfortably monitor 50-100 sports events with 5-10 markets each — that is 250-1000 markets, well within the API rate limits. To scale to thousands of markets per day (the full Polymarket sports calendar across NBA, NFL, MLB, NHL, soccer, tennis, MMA), you need a sharded architecture.

Sharding by sport/league

The simplest sharding strategy is by sport: one scanner instance per sport (NBA scanner, NFL scanner, soccer scanner, etc.). Each instance has its own API rate limit budget, its own VPS, and its own bankroll allocation. Coordination is minimal — the only shared state is the blacklist and the exposure tracker, both of which can be backed by Redis.

Async I/O vs Celery

The scanner's main loop is async-first (asyncio), which is optimal for I/O-bound work (HTTP requests, WebSocket messages). For CPU-bound work — specifically the LP solver in optimizer.py — offload to a Celery worker pool backed by Redis or RabbitMQ. The scanner's detector publishes LP-solve tasks to a Celery queue; the worker pool solves them in parallel and returns the optimal allocation. This keeps the event loop responsive while still solving LPs in parallel across CPU cores.

Hot/cold market tiering

Not all markets deserve the same scan frequency. Tier markets by activity: hot markets (in-play, or about to resolve within 1 hour) get WebSocket subscriptions with sub-second updates; warm markets (resolving within 12 hours) get 5-second polling; cold markets (resolving in 1-7 days) get 60-second polling. This tiering reduces API load by 80%+ compared to uniform high-frequency polling of all markets.

Tier	Definition	Transport	Frequency	Execution
Hot	In-play or resolves <1h	WebSocket	Sub-second	Maker + taker
Warm	Resolves 1-12h	REST poll	5 seconds	Maker preferred
Cold	Resolves 1-7 days	REST poll	60 seconds	Maker only

Tier	Definition	Transport	Frequency	Execution
Stale	No trades in 1h+	Skip	—	—

Table 4.3 — Market tiering for scan frequency

4.6 Legal and compliance

Polymarket operates in a regulatory gray area. The platform is technically available only to non-US users, but US users can access it via VPN. Whether this is legal in your jurisdiction is a question for a lawyer, not this guide. The compliance notes below are operational best practices, not legal advice.

USDC custody on Polygon

All trading capital lives as USDC on the Polygon blockchain. Wallet security is therefore paramount. Recommended setup: a hardware wallet (Ledger or Trezor) holds the treasury; a hot wallet with limited funds (1-5% of treasury) handles active trading. Refill the hot wallet from the hardware wallet weekly. Never store the private key in plain text on the VPS — use environment variables loaded from an encrypted secrets manager (AWS Secrets Manager, HashiCorp Vault, or age-encrypted files at rest).

Tax lot tracking

Every fill is a taxable event in most jurisdictions. The US treats prediction market gains as short-term capital gains (ordinary income rate) regardless of holding period, because the contracts are considered Section 1256 contracts in some interpretations and gambling winnings in others. The exact treatment is genuinely uncertain — consult a tax professional. The scanner should log every fill with timestamp, price, shares, and fee, and export to CSV for accountant review at year-end.

Wallet security best practices

- Never expose POLY_PRIVATE_KEY in code, logs, or shell history. Use environment variables loaded from an encrypted source.
- Use a dedicated trading wallet, separate from your main Ethereum wallet. If the trading wallet is compromised, only the trading capital is at risk.
- Set up transaction alerts on the trading wallet — every outgoing transaction triggers a Telegram notification. If you see a transaction you did not authorize, you can react within seconds.
- Use a multi-sig wallet (Gnosis Safe) for the treasury. Require 2-of-3 signatures for any withdrawal over $10,000. This prevents a single compromised key from draining the treasury.

CHAPTER 5 · STRATEGY & MAINTENANCE

Edge Maintenance & Failure Modes

Arbitrage edges decay. As more operators build scanners and compete for the same opportunities, the gross edge per trade shrinks. This chapter covers why sports has stayed inefficient longer than politics, the common failure modes that take down amateur scanners, hybrid strategies that combine Polymarket arbitrage with other edges, and a roadmap for evolving the scanner as the market matures.

5.1 Why sports stays inefficient

The article framing this guide makes the case clearly: sports stays inefficient because of who trades it. Politics arbitrage gets picked over by smart money — quants, funds, and bot squads all camp the election. Sports gets retail. People bet their team with their heart. They pile into a YES on emotion the second their guy scores. Then an injury hits, a ref call lands, momentum flips, and the price lags the new reality for a few seconds while the order book catches up.

This compositional difference is structural, not transient. The smart money that does trade sports tends to focus on high-stakes events (Super Bowl, NBA Finals) where liquidity supports large positions. The nightly NBA, NFL, and MLB regular-season games are left to retail flow and a handful of small-to-mid arbitrage operators. As long as retail flow dominates these markets, the inefficiency will persist.

Sources of sports inefficiency

Beyond retail emotion, several structural factors keep sports markets inefficient:

- **Recency bias on injuries** — when a star player is announced as injured 30 minutes before tipoff, the sportsbooks re-price within seconds but Polymarket often lags by 30-60 seconds because the order book is thinner.
- **Lag between sportsbook line moves and Polymarket re-pricing** — Pinnacle and Circa move their NBA lines constantly based on sharp action; Polymarket prices follow with a 5-30 second delay. This is the cleanest cross-platform edge.
- **Emotional betting on popular teams** — Lakers, Cowboys, Yankees games see disproportionate YES volume on the popular team regardless of true odds. The overvalued side is consistently the popular team.
- **In-play momentum shifts** — a 10-0 run in basketball creates an emotional cascade of bets on the leading team, driving the price 5-10 cents above fair value for 30-60 seconds.

5.2 Common failure modes

Most amateur arbitrage scanners fail in predictable ways. The failure modes below are ordered by frequency observed in the wild.

Failure 1: Leg execution failure

Symptom: One leg of a multi-leg arb fills, the other does not, leaving unhedged directional exposure. **Cause:** Order book moves between leg submission and fill, or one leg has insufficient depth. **Mitigation:** Concurrent leg placement (Section 3.8), partial-fill hedging (Section 3.8), and rollback on critical failure. Pre-trade depth check should require both sides to have at least 2× the target notional in the top 5 levels.

Failure 2: Resolution disputes

Symptom: A market you bet on enters UMA dispute and resolves against you, turning a guaranteed win into a loss. **Cause:** Ambiguous resolution criteria — rain-delayed MLB games, NFL overtime edge cases, markets where the question text is imprecise. **Mitigation:** Blacklist markets with prior disputes; avoid markets with ambiguous resolution language; size positions with a 5-10% settlement-risk haircut.

Failure 3: Liquidity mirage

Symptom: The detector reports a deep book, but your order only fills 20% before the book vanishes. **Cause:** The top-of-book was a single large order that got pulled the moment you started filling against it — either a market maker responding to your order, or a spoofing order that was never intended to fill. **Mitigation:** Use the depth-weighted slippage estimator (Section 2.5), not just top-of-book. Reject opportunities where the top 5 levels do not have at least 2× your target notional.

Failure 4: Fee threshold miscalibration

Symptom: The scanner reports profitable trades but the bankroll declines over time. **Cause:** The min-edge threshold is set below the actual all-in cost (fee + slippage + settlement risk), so the scanner is taking trades with negative expected value. **Mitigation:** Track realized PnL per trade against detected edge; if realized < detected by more than 30 bps consistently, raise the threshold.

Failure 5: NegRisk settlement bugs

Symptom: A NegRisk multi-outcome market settles in an unexpected way, leaving your covering portfolio with a hole. **Cause:** NegRisk resolution logic is more complex than binary markets; edge cases around tie-breaking, withdrawal, and dispute resolution can produce surprising outcomes. **Mitigation:** Treat NegRisk arbitrage as Category 3 (edge-adjusted, not riskless) and size with full Kelly haircut. Avoid NegRisk markets with more than 12 outcomes — settlement complexity scales super-linearly.

Failure 6: Hot wallet drainers

Symptom: The trading wallet is drained to zero with no fills in the scanner log. **Cause:** The private key was compromised — typically via a phishing attack on the operator, a leaked environment variable, or a malicious dependency in pip install. **Mitigation:** Hardware wallet for treasury, hot wallet with limited funds, transaction alerts, multi-sig for large withdrawals. Audit requirements.txt dependencies regularly with pip-audit.

Failure 7: API rate limit cascades

Symptom: The scanner enters a retry loop, hits rate limits on retries, and effectively shuts down for minutes at a time. **Cause:** A transient API error triggers retries, which themselves trigger more retries, which exhaust the rate limit. **Mitigation:** Exponential backoff with jitter, circuit breaker that pauses scanning for 60 seconds after 5 consecutive failures, and rate limit budget tracking that throttles new requests before hitting the limit.

5.3 Hybrid strategies

Pure Polymarket arbitrage is one edge among many. The most profitable operators combine it with related strategies that share infrastructure but harvest different inefficiencies. Below are three hybrid strategies worth considering once the core scanner is stable.

Polymarket arb + sportsbook promo arb

Traditional sportsbooks offer deposit bonuses, free bets, and odds boosts as acquisition tools. These promotions often have positive expected value on their own, but combining them with Polymarket arbitrage creates a higher-confidence edge. For example: a sportsbook offers a $500 free bet on the Lakers moneyline at +150 (post-promo); Polymarket has Lakers YES at $0.55. Take the free bet on the sportsbook (expected value ≈ $300), and hedge by buying Celtics YES on Polymarket for $275. Either outcome: Lakers win → sportsbook pays $750, Polymarket loses $275, net $475; Celtics win → sportsbook loses nothing (free bet), Polymarket pays $500, net $225. Expected value ≈ $350, risk-free.

Parlay insurance farming

Many sportsbooks offer parlay insurance — if one leg of a 5-leg parlay loses, the stake is refunded as a free bet. The strategy: construct a 5-leg parlay where 4 legs are high-probability and the 5th is a low-probability longshot that you actively want to lose. If the longshot loses (high probability), you get the stake back as a free bet to deploy on Polymarket arbitrage. If the longshot wins, you collect the parlay payout (which is large). Either outcome is positive expected value, and the capital cycles back into your Polymarket scanner.

Free-bet conversion

Free bets on sportsbooks typically pay only the winnings, not the stake. The optimal strategy is to deploy free bets on longshot markets (high decimal odds) to maximize expected value. Polymarket arbitrage complements this by providing a place to deploy the converted free-bet winnings at risk-free rates while waiting for the next sportsbook promotion.

5.4 Edge decay monitoring

Every arbitrage edge decays over time as more operators compete for it. The scanner should track per-strategy performance over rolling 30-day windows and alert when a strategy's Sharpe ratio drops below a kill threshold. Recommended thresholds:

Strategy	Healthy Sharpe	Kill threshold	Action on kill
Single-market (maker)	1.5	1.0	Raise min_edge_maker_bps by 5 bps
Single-market (taker)	1.0	0.5	Switch fully to maker; consider pausing taker
NegRisk rebalance	0.8	0.3	Pause; only run during event windows
Combinatorial (LP)	1.2	0.7	Recheck feasibility constraints; tighten LP
Cross-platform	1.0	0.5	Recalibrate sportsbook vig assumption

Table 5.1 — Edge decay monitoring thresholds (rolling 30-day Sharpe)

When a strategy hits its kill threshold, do not delete it — just disable it temporarily and recheck weekly. Edges sometimes recover when competitor bots get redeployed elsewhere or when market structure shifts. A strategy that was killed in March might be profitable again in October.

5.5 Roadmap

The scanner described in this guide is a starting point. Three directions worth exploring once the core system is stable:

ML-based dependency discovery

The current combinatorial detector uses hand-written regex patterns to classify markets by category (moneyline, spread, total, player prop) and hand-coded feasibility constraints to prune impossible atomic outcomes. A production system should replace this with a fine-tuned language model that reads market questions and outputs structured (category, team_home, team_away, spread_value, total_value) tuples. This unlocks combinatorial arbitrage on novel market types that the regex patterns miss.

Market-making hybrid

Pure arbitrage captures the spread when it inverts, but most of the time the spread is positive and you are flat. A market-making layer can capture the spread continuously by resting limit orders on both sides of normal (non-inverted) markets, earning the maker fee (zero) and capturing the spread when both sides fill. This is a different risk profile — you carry inventory risk — but it dramatically increases capital efficiency. The infrastructure (order book monitoring, limit-order placement, position tracking) is identical to the arbitrage scanner.

Cross-venue HFT

The cross-platform detector in this guide is a low-frequency tool — it compares Polymarket prices to sportsbook odds every 30-60 seconds. A production cross-venue system would subscribe to a sportsbook odds feed (Pinnacle's API, for example) and the Polymarket WebSocket simultaneously, executing within milliseconds of a line move. This requires significant infrastructure (co-located servers, sportsbook API access, capital on both venues) but captures the cleanest edge in the strategy space.

5.6 Closing thoughts

The edge in Polymarket sports arbitrage is real, repeatable, and accessible to operators with moderate engineering resources. The 2026 fee change made the game harder, but the maker-fee loophole preserves the edge for operators willing to do the operational work of resting limit orders and managing fill risk. The nightly cadence of professional sports produces a fresh batch of opportunities every day, unlike the seasonal cadence of politics that leaves capital idle for months at a time.

The wallets quietly printing month after month on Polymarket are pointed at the games, not the elections. Point your scanner where the games are.

APPENDIX A · CODE PACKAGE

Code Module Index

The companion Python package contains twelve runnable modules totaling roughly 2,500 lines of code. This appendix lists each module, its purpose, key public functions, and dependencies. All modules live in the arb_scanner/ directory of the deliverable.

Module	Purpose	Key public API	Dependencies
config.py	Configuration dataclasses + env/YAML loaders + fee constants	load_config(), taker_fee_rate(), maker_fee_rate()	PyYAML
models.py	Domain dataclasses: Market, OrderBook, Opportunity, Position, etc.	Market, OrderBook, Opportunity, Leg, Position, AtomicOutcome	—
gamma_client.py	Polymarket Gamma API client (events, markets, metadata)	GammaClient, detect_sport(), detect_category()	httpx
clob_client.py	CLOB REST + WS clients (order books, prices, order placement)	CLOBRestClient, CLOBWSClient, CLOBOrderClient	httpx, websockets, py-clob-client
detectors.py	Opportunity detectors: 4 strategy types	detect_single_market_arb(), detect_negrisk_rebalance_arb(), detect_combinatorial_arb(), detect_cross_platform_arb()	models, config
optimizer.py	PuLP LP solver for combinatorial covering portfolios	solve_covering_lp(), greedy_cover_heuristic(), is_feasible_outcome()	pulp
risk.py	Kelly sizing, exposure tracker, blacklist, slippage estimator	conservative_kelly_size(), risk_gate(), ExposureTracker, Blacklist	models, config
executor.py	Concurrent leg execution, partial-fill hedging, rollback	Executor, LegExecutionResult, rollback_position()	clob_client, models
alerts.py	Telegram + Discord notifications	AlertSender (send_opportunity_alert, send_fill_alert, send_error_alert)	httpx
scanner.py	Main scanner loop tying everything together	Scanner, setup_logging()	all other modules
backtest.py	Historical backtest framework + snapshot recorder	run_backtest(), load_snapshots(), record_snapshots()	detectors, models
main.py	CLI entry point (scan, backtest, record, test-api, show-config)	main()	scanner, backtest, config

Table A.1 — Code module index

A.1 Installation

▸ **Installation**

```
$ cd arb_scanner/
$ python -m venv venv &amp;amp;&amp;amp; source venv/bin/activate
$ pip install -r requirements.txt

# Verify installation
$ python main.py show-config

# Test API connectivity (no auth needed)
$ python main.py test-api
```

A.2 Environment variables

▸ **Environment variables**

```
# Required for live trading
export POLY_API_KEY="..."
export POLY_API_SECRET="..."
export POLY_API_PASSPHRASE="..."
export POLY_PRIVATE_KEY="0x..."
export POLY_WALLET_ADDRESS="0x..."

# Optional: faster Polygon RPC
export POLY_RPC_URL="https://polygon-mainnet.g.alchemy.com/v2/YOUR_KEY"

# Optional: alerts
export POLY_TG_TOKEN="..."
export POLY_TG_CHAT_ID="..."
export POLY_DISCORD_WEBHOOK="https://discord.com/api/webhooks/..."

# Optional: override defaults
export POLY_BANKROLL_USDC=50000
export POLY_MIN_EDGE_TAKER_BPS=75

# Optional: YAML config file
export POLY_CONFIG_PATH="./config.yaml"
```

A.3 requirements.txt

▸ **requirements.txt**

```
# Polymarket Sports Arbitrage Scanner — Dependencies
# Tested with Python 3.11+

# Core async HTTP / WS
httpx&gt;=0.27.0
websockets&gt;=12.0

# Configuration
PyYAML&gt;=6.0

# Polymarket official SDK (order placement, signing)
py-clob-client&gt;=0.20.0

# Linear programming (combinatorial arbitrage)
pulp&gt;=2.7.0
```

APPENDIX B · REFERENCE

Mathematical Reference

Condensed formula sheet for quick reference. Each formula below is used in the production scanner; the section references point back to the full derivation in Chapter 1.

B.1 Implied probability & overround

Given YES ask P_yes and NO ask P_no:

$$p\_implied(YES) = P\_yes$$

$$p\_implied(NO) = P\_no$$

$$overround = (P\_yes + P\_no) - \$1$$

$$p\_norm(YES) = P\_yes \,/\, (P\_yes + P\_no)$$

$$p\_norm(NO) = P\_no \,/\, (P\_yes + P\_no)$$

B.2 Single-market arbitrage

Buy 1 YES at ask + 1 NO at (1 − YES bid). Arbitrage condition (post-fee):

$$(P\_yes\_ask + P\_no\_ask) \cdot (1 + f) < \$1$$

Equivalently:

$$P\_yes\_ask + P\_no\_ask < 1 \,/\, (1 + f)$$

At f = 0.0075, threshold ≈ $0.9926. Profit per share pair:

$$\pi = \$1 - (P\_yes\_ask + P\_no\_ask) \cdot (1 + f)$$

Maker execution (fee = 0): rest both sides at limit; if both fill:

$$\pi\_maker = \$1 - (P\_yes\_rest + P\_no\_rest)$$

B.3 NegRisk multi-outcome rebalance

For N-outcome NegRisk market. Buy-all-YES arb condition:

$$\Sigma\_i \, P\_yes\_ask(i) \cdot (1 + f) < \$1$$

Buy-all-NO arb condition (equivalent to sell-all-YES):

$$\Sigma\_i \, P\_yes\_bid(i) > \$1 + fee\_buffer$$

Buy-all-NO payout: (N − 1) · $1, because all NOs except the winner's pay out.

B.4 Combinatorial covering portfolio (LP)

Variables: x_p ≥ 0 for each position p. Payoff matrix A[j, p] ∈ {0, 1}. Cost c_p per position.

$$\text{minimize } \Sigma_p \, c_p \cdot x_p$$

$$\text{subject to } \Sigma_p \, A[j, p] \cdot x_p \geq 1 \; \forall \text{ atomic outcomes } j \in J$$

$$x_p \geq 0 \; \forall \text{ positions } p$$

Arbitrage exists if optimal C* < $1 (after fee adjustment).

Dual interpretation: y_j ≥ 0 is a probability distribution over outcomes. By strong duality, if C* < $1 then no feasible probability distribution makes the market expected-value-neutral.

B.5 Cross-platform arbitrage

Decimal odds d imply probability 1/d. American odds +a imply 100/(a+100); −a imply a/(a+100). Vig-free normalized probability:

$$p_{vigfree}(YES) = p_{implied}(YES) \,/\, (p_{implied}(YES) + p_{implied}(NO))$$

Edge:

$$edge = p_{vigfree}(YES) - P_{yes\_polymarket}$$

$$net\_edge = edge - taker\_fee_{polymarket} - sportsbook\_vig/2$$

B.6 Kelly criterion

Full Kelly fraction of bankroll:

$$f^* = (p \cdot b - q) \,/\, b$$

where p = win probability, q = 1 − p, b = net odds (decimal − 1).

Conservative arbitrage Kelly (with settlement-risk haircut):

$$f_{cons} = kelly\_fraction \cdot kelly(0.95,\ 1 + edge)$$

Position size = bankroll × f_cons, capped at kelly_cap_bps and floored at kelly_floor_bps.

B.7 Fee-adjusted break-even

Break-even edge (gross bps) required to net zero profit after fees:

$$break\_even\_bps = f \cdot 10\_000 \cdot (n\_legs)$$

where n_legs is the number of taker legs (each pays fee). For 2-leg taker arb at 75 bps fee:

$$break\_even\_bps = 0.0075 \cdot 10\_000 \cdot 2 = 150 \text{ bps}$$

For 2-leg maker arb at 0 bps fee:

$$break\_even\_bps = 0 \cdot 10\_000 \cdot 2 = 0 \text{ bps}$$

B.8 Odds conversion quick-reference

Decimal	American	Implied prob	Polymarket $
Decimal	American	Implied prob	Polymarket $
1.20	−500	0.833	$0.83
1.50	−200	0.667	$0.67
1.91	−110	0.524	$0.52
2.00	+100	0.500	$0.50
2.50	+150	0.400	$0.40
3.00	+200	0.333	$0.33
5.00	+400	0.200	$0.20
10.00	+900	0.100	$0.10

Table B.1 — Odds format conversion reference

B.9 Slippage model

Volume-weighted average price (VWAP) for buying S shares against the ask side:

$$VWAP = (\Sigma\_k\ p\_k \cdot min(s\_k,\ S\_remaining)) \ / \ S$$

where (p_k, s_k) are price/size at ask level k. Slippage in bps:

$$slippage\_bps = (VWAP - best\_ask) \ / \ best\_ask \cdot 10\_000$$